

基于价格数据与抽样检测的生产流程检验与拆解决策设计

摘要

本文根据某企业生产电子产品实际情况，建立了基于动态规划思想的数学模型，设计在生产过程中对零配件、半成品、成品进行适当检测和拆解的决策方案，帮助企业提升产品质量的同时实现收益的最大化。

针对问题一，首先考虑到检测成本，所以我们采用**单次抽样计划**。由于样本总量很大，可近似认为次品数服从**二项分布**，利用描述该计划性能的**OC 曲线**上的特征点，建立样本量与样本中残次品总量的二元方程。之后利用**模拟退火算法**，通过设定初始温度、规定迭代次数，接受较差结果的概率遵循**Metropolis 准则**，连续运算多次，取运算结果的平均值并进行取整，得到最终的结果。

针对问题二，首先要对产品生产的各个阶段做出决策。使得成本最小化而利润最大化，引入二进制决策变量 x, y 表示是否检测和是否拆解的不同决策。结合实际，我们假设该生产线很稳定，每批产品中成品无论好坏，数量均为定值 N 。在此基础上，运用**动态规划**的思想，反推出零配件的数量，同时，引入决策变量 x 修正产品被检测而带来次品率的变化。对于成品是否拆解，通过**贝叶斯公式**计算拆解得到零配件的**价值期望**并与拆解费用进行比较，从而决策拆解对应决策变量 y 的取值。拆解所得的零配件可以重复利用，将作为回收收益计入利润中，最终利润 W 为销售额加回收收益减去各类成本。由于决策变量较少，仅 4 个，我们通过**穷举法**求解模型，得到最佳决策方案。

针对问题三，由于 m, n 不确定，故先从 $m = 2, n = 8$ 的特例入手，沿用问题二的思路，同样引入决策变量确定各类成本及收益。注意到半成品 1 和 2 生产涉及到相关参数的一致性，可以把半成品 2 用半成品 1 **等效替代**，从而减少决策变量的数量，简化计算。最终我们得到 11 个决策变量，采用**穷举法**和**遗传算法**两种方法进行求解，得到最佳决策方案，成品利润可以达到 **138.71 元/件**。对于 m, n 未知的一般情况，考虑到生产过程具有多阶段的特点，可以用**动态规划**模型，并使用**启发式算法**降低计算复杂度。

针对问题四，假设商家声称的次品率标称值和真实值之间的差距较小，且真实次品率落在以标称值为中心的区间上。采用**贝叶斯估计**逼近真实次品率时，首先确定 **Beta 分布**中 α 和 β 的值，完成信息先验，再设定真实次品率可能存在的区间，从区间内取定合理步长，基于**贝叶斯定理**模拟抽样二次分布的数据，修正次品率得到后验分布，接着用**蒙特卡洛模拟**生成样本，循环评估后验分布的特性，不断修正估计值，使其更接近于真实值，最后生成标准正态分布的点，计算一次抽样检测的次品区间，抽样并计算最终次品率，带入问题二和问题三计算，得到结果。

关键字：OC 曲线 穷举法 动态规划 贝叶斯估计 启发式算法 蒙特卡洛模拟

一、问题重述

1.1 问题背景

企业生产的某种电子产品十分畅销。为响应市场需求、提升企业生产利润，企业决定优化产品检测工序。生产该电子产品需要购买两种零配件，然后企业将这两种零配件装配成成品。在装配时，任意零配件不合格，则成品一定不合格；即使两种零配件均合格，成品也不一定合格。对于不合格成品，企业可以选择报废或拆解的方式进行处理，虽然报废不会损坏零配件，但需要支付相应费用。

1.2 问题提出

企业要求根据生产的相关数据，建立相应数学模型解决以下问题：

问题一：需针对以下情形，设计检测次数尽可能少的抽样检测方案来给出结果：

- (1) 在 95% 的信度下认定零配件次品率超过 10%，则拒收该批零配件；
- (2) 在 90% 的信度下认定零配件次品率不超过 10%，则接收这批零配件。

问题二：需根据两种零配件和成品次品率，对于这种电子产品生产的各个阶段（零配件检测、成品装配、不合格成品处理）和投入市场阶段做出决策。根据所做决策，针对表 1 中的情形给出具体的决策方案，并给出相应依据及指标结果。

问题三：需根据零配件、半成品和成品的次品率，参照问题 2，对于 m 道工序、 n 个零配件，给出生产过程的决策方案。根据组装情况和表 2 中的情形，给出具体的决策方案，以及决策的依据及相应指标。注意到

问题四：需重新完成问题 2 和问题 3，规定两个问题中的零配件、半成品和成品的次品率均是通过抽样检测方法得到的。

二、问题分析

2.1 问题一：抽样检测方案设计

问题一需设计尽可能抽样检测次数少的检测方案，针对两种情形，决定是否接收零配件。首先我们使用中国现行国家标准¹提供的抽样计划，为了使抽样检测次数尽可能少，我们选择单次抽样计划。然后将 OC 曲线²上的特征点代入其表达式。[3] 最后通过模拟退火算法得到两种情形下的样本量和次品率，从而得到具体结果。

¹GBT 2828.1-2012 计数抽样检验程序第 1 部分：按接收质量限 (AQL) 检索的逐批检验抽样计划

²操作特性曲线 (Operating Characteristic curve), 主要用于质量控制中的抽样检验，表示在不同的不合格品比例下，某个抽样方案通过的概率。

2.2 问题二：1 道工序，2 个零配件的决策方案设计

问题二要求对产品生产的各个阶段做出决策，使得售出这一批产品的成本最小，利润最大。需要计算流程中每个过程的成本与收益，并引入二进制变量 x, y 来对具体的检测以及拆解方案进行决策。最后将题中的具体的决策方案的数据代入模型求解。

2.3 问题三： m 道工序， n 个零配件的决策方案设计

问题三在问题二 1 道工序，2 个零配件的基础上进一步拓展为 2 道工序，8 个零部件和 m 道工序， n 个零部件的情况。

对于前一种情况，需要对问题二计算流程中每个过程的成本与收益进行相应的修改，注意到到半成品 1 与半成品 2 的装配所需的零配件的装配成本及检测成本、装配成本、检测成本、次品率等参数都相同，因此二者等价，相应的决策变量可以合并，以简化求解难度。

对于后一种情况，由于不确定因素过多，无法给出确定的方案，由于生产过程具有多阶段的特点，可以考虑通过动态规划模型，并使用启发式算法进行求解。

2.4 问题四：真实次品率的修正

问题四中问题二三中给定的次品率标称值为抽样检测的频率，要求修正使其更接近真正的概率。需要通过增加抽样次数的方法修正概率的估计值。考虑到真实概率不会偏离给定的标称值太远，综合考虑时间和检测成本，我们决定重新抽取一次对之前的概率进行修正，并将修正后的概率代入问题二三中重新计算。

三、模型的假设

本文基于电子产品的生产情况提出以下合理假设：

- 半成品、产品的次品率是将正品零配件（或半成品）装配后的产品次品率；
- 检验的批次是均匀的，同一批次的各产品应来自于相同的机器、作业员、原料，而且几乎是在同一时间生产的；
- 零配件足够多，样本总体足够大；
- 产品包装完好，没有在运送途中受到损害；
- 不同零配件、不同半成品、不同成品装配的过程均相互独立，互不干扰；
- 商家声称的次品率标称值和真实值之间的差距较小。

四、符号说明

表 1 中显示的是本文章中的主要符号及定义和单位，文中出现的其他符号和具有角标的表中符号会在上下文中说明具体含义

表 1 主要符号及定义

符号	定义	单位
n	样本量	件
d	样本中的残次品总量	件
e	样本中允许存在的最大残次品总量	件
p	次品率	-
P_a	接收概率	-
β	消费者风险，即接收质量不合格批次的概率	-
α	生产者风险，即拒绝质量合格批次的概率	-
N	装配完成的全部成品或半成品数 ³	件
a	购买单价	元
c	各类成本	元
P	总合格率	-
s	成品的市场售价	元
x	决策变量，是否检测产品	-
y	决策变量，是否拆解次品	-

五、问题一模型的建立与求解

5.1 模型的建立

5.1.1 建立模型的准备

在问题一建立模型的准备中，主要任务是推导 OC 曲线表达式。OC 曲线为操作特性曲线 (Operating Characteristic Curve), 主要用于质量控制中的抽样检验，表示在不同的不合格品比例下，某个抽样方案通过的概率。

OC 曲线表达式如下：

$$P_a = P(d \leq e) = \sum_{d=0}^e C_n^d p^d (1-p)^{n-d} \quad (1)$$

³—轮完整的生产过程所产生的所有成品或半成品数量, 包括不合格产品, 但不包括拆解后重新装配的成品或半成品

曲线上点记为 (p, P_a) ，其中 p 表示待检验的零配件批次的次品率， P_a 表示接收概率，即在给定的抽样方案下，批次被接受的机会大小。 d 表示样本中的残次品总量， e 表示样本中允许存在的最大次品总量， n 表示抽取的样本数量。

OC 曲线表达式的推导如下：

假设：在总体为 N 的样本中，有 M 件次品，抽取 n 件样品，并设抽到 X 件次品。则次品数 X 服从超几何分布 $H(n, M, N)$ ，表示为 [1]：

$$P(X = k) = \frac{C_M^k C_{N-M}^{n-k}}{C_N^n}, \max\{0, n - N + M\} \leq k \leq \min\{n, M\} \quad (2)$$

为了便于计算与后续处理，并结合实际，我们假设样本总体足够大，此时 X 可近似认为服从二项分布 $B(n, p)$ ，其中 n 为抽取的样品数， $p = \frac{M}{N}$ 为单个零配件是次品的概率。则有

$$P(X = d) = C_n^d p^d (1 - p)^{n-d}, d = 0, 1, 2, \dots, n, 0 < p < 1 \quad (3)$$

其中， d 表示样本中实际抽取到的次品数量， C_n^d 是二项系数，是从 n 个样本中选取 d 个次品的组合数。当样本中的次品数 $d \leq e$ 时，该批次被接受，即：

$$P_a = P(X \leq e) = \sum_{k=0}^e P(X = k) \quad (4)$$

将上式代入二项分布的表达式，可得到 OC 曲线的表达式。

图 1 为 OC 曲线的示意图， $(AQL, 1-\alpha)$ 和 $(LTPD, \beta)$ 为 OC 曲线上的特殊点，AQL 表示接受水平，即买方（企业）认为满意的产品品质水准。 α 为生产者风险。 $(AQL, 1-\alpha)$ 特殊点表示当产品的次品率为 AQL 时，接受产品的概率为 $1-\alpha$ 。

LTPD 表示拒收品质水平，即买方（企业）认为产品品质已经差到应该直接拒收的水平。 β 为消费者风险。 $(LTPD, \beta)$ 特殊点表示当产品的次品率为 LTPD 时，接收产品的

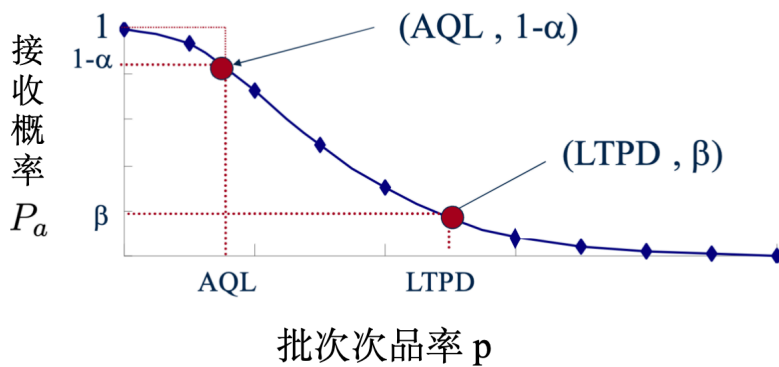


图 1 OC 曲线的示意图

概率为 β 。

5.1.2 设计抽样检测方案

GB/T 2828.1-2012 是中国国家标准，将 AQL（可接受质量水平）作为检索指标，包含了为不同 AQL 和检验水平提供的样本量以及接受/拒绝标准的表格。主要应用在工业产品的计数抽样检验，包括机械制造、电子产品、消费品等行业。这是一种成本较低但有效的质量控制方法，能帮助电子企业平衡质量控制成本和质量保障之间的关系。

由于题目没有给定待检验的零配件总量，我们无法直接由中国现行国家标准来确定抽取的样本量。但是它仍然能通过给定的 AQL 和检验水平为抽样计划提供指导。首先根据中国现行国家标准，按照给定的情形确定抽样计划。

对于情形 1——在 95% 的信度下认定零配件次品率超过 10%，则拒收该批零配件，抽样计划如下：

1. 为尽量使抽样检测次数减少，采用单次抽样计划
2. AQL(可接受质量水平): 10%（表示次品率低于 10% 的批次可以接受）
3. 接受概率：当批次的缺陷率为 10% 时，批次被接受的概率为 95%

从给定的条件可知，当情形 1 采用单次抽样计划时，其 OC 曲线上的特殊点（AQL， $1-\alpha$ ）即为（0.1，0.95），将该点代入 OC 曲线表达式，可得下式：

$$0.95 = \sum_{d=0}^{\lfloor \frac{n}{10} \rfloor} C_n^d 0.1^d 0.9^{n-d} \quad (5)$$

对于情形 2——在 90% 的信度下认定零配件次品率不超过 10%，则接收这批零配件，抽样计划如下：

1. 为尽量使抽样检测次数减少，采用单次抽样计划
2. LTPD（拒收品质水平）：10%（表示缺陷率高于 10% 的批次拒绝接受）
3. 接受概率：当批次的缺陷率为 10% 时，批次被接受的概率为 90%

从给定的条件可知，当情形 2 采用单次抽样计划时，其 OC 曲线上的特殊点（LTPD， β ）即为（0.1，0.1），将该点代入 OC 曲线表达式，可得下式：

$$0.1 = \sum_{d=0}^{\lfloor \frac{n}{10} \rfloor} C_n^d 0.1^d 0.9^{n-d} \quad (6)$$

对于(5)式和(6)式，其中 n 和 d 均为自然数。

5.2 基于模拟退火算法的抽样检测模型求解

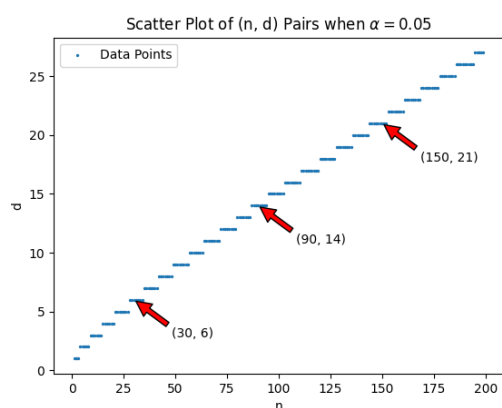
本题中 n 、 d 是离散的整型变量。针对整型优化问题，很多常规的优化方法（如梯度法、线性规划）不适用，而模拟退火算法 (Simulated Annealing, SA) 可以很好地处理离散整型变量的优化问题。模拟退火不要求变量的连续性，因此它可以通过随机地在离散范围内生成新解来优化。

模拟退火算法的主要原理如伪代码 Algorithm1 所示。从伪代码中可知，模拟退火中存在随机性，存在从局部最优中跳出的可能性；并且跳出概率依赖于设定温度，在过程中不断变小，可以保证收敛于最优值。在模拟退火算法中我们规定温度需要迭代 1000 次，同时使接受较差的结果的概率遵循 Metropolis 准则。我们使用模拟退火算法连续运算 1000 次，取每次运算最优 n 、 d 的平均值然后进行取整操作，最终得到的结果如表2所示：

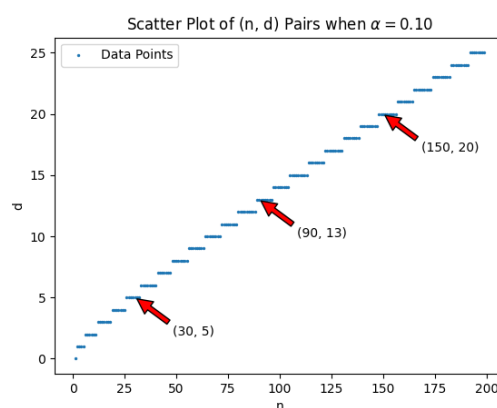
表 2 问题一的最优样本量

情形	n	d	次品率
情形 1	43	4	9.30%
情形 2	53	3	5.67%

两种情形的次品率均低于 10%，因此公司可以接受这两种情形下的零配件批次。两种情形下的样本量 n 与样本中可容纳的次品数 d 的关系如图2a2b所示。从这两个图中，我们可以知道当标称值和抽取的样本量相同时，显著性水平越大，样本中可容纳的次品量越少。



(a) $\alpha = 0.05$ 时， n 与 d 的关系图



(b) $\alpha = 0.10$ 时， n 与 d 的关系图

图 2 抽取的样本量和样本中可容纳次品数在不同显著性水平下的关系图

Algorithm 1: 模拟退火算法优化 n 和 d 的求解

Input: 温度, 迭代次数, 冷却速率等模拟参数**Output:** 最佳样本量 n 和样本中次品量 d 及其误差

```
1 随机初始化  $n$  和  $d$  的值;  
2 for 温度不断迭代 do  
3   for 同一温度下的每次迭代 do  
4      $n$  随机加上一个新值  $\epsilon$ , 根据新  $n$  得到  $d$  的值;  
5     if 新的  $n$  对应的累积概率小于原概率 then  
6        $n$  被新  $n$  替代;  
7     else  
8        $n$  有一定概率 (与差值相关) 被新  $n$  替代;  
9   更新温度与暂存  $n$  值;  
10 return 最佳  $n$  和  $d$  及其误差;
```

六、问题二模型的建立与求解

6.1 模型的建立

在工业生产中, 如果流入市场的次品过多, 会极大地损坏企业的信誉并且影响产品的销量。但是, 如果一味地追求低次品率, 检测费用会极大地压缩产品的盈利空间。考虑到不合格成品中的调换损失包括了物流成本、企业信誉等损失, 我们将该电子产品的总盈利作为决策的指标。

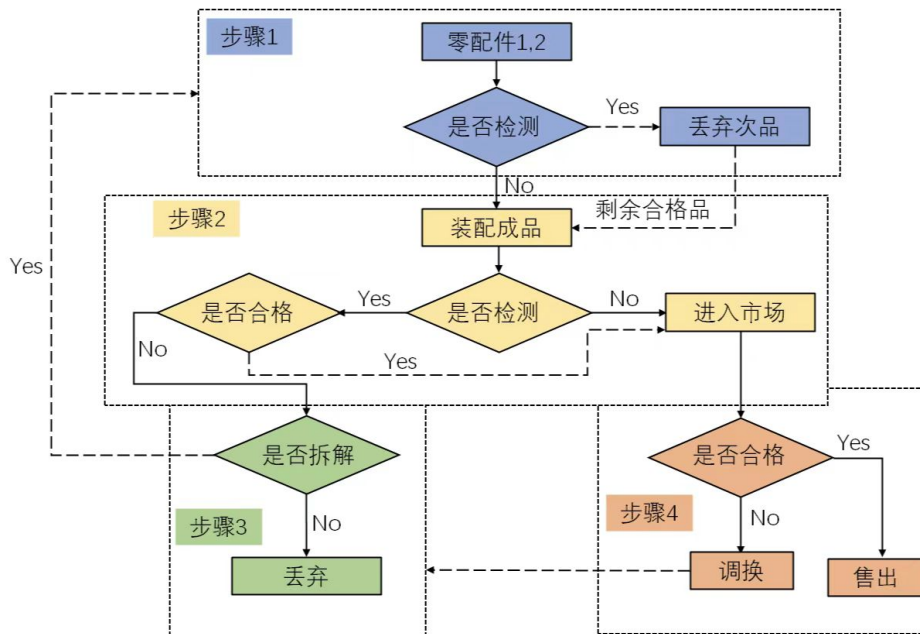


图3 问题二生产过程决策流程图

由题中条件, 我们可以将总盈利视为总销售额与总成本之差。其中成本来自于以下

步骤：零配件 1、2 的采购与检测，成品的装配、检测成本及调换损失，拆解费用。在生产过程中，我们不妨假设只对同一不合格成品进行至多一次拆解，理由如下：假设被拆解的不合格成品数量为 n ，如果拆解它们得到的零配件再次组装，与前一次相同的两零配件组装到一起的概率为：

$$p_{same} = \frac{n}{n^2(n-1)^2 \dots 1^2} = \frac{n}{(n!)^2} \quad (7)$$

显然此概率 p_{same} 足够小，因此可以认为只对同一不合格成品进行至多一次拆解。

问题 2 涉及到零配件 1,2 与成品，我们用 i 取 1,2,3 分别表示零配件 1、零配件 2、成品。对于决策变量, x_i 表示是否检测, y 表示是否拆解不合格成品，取值有 [4]

$$x_i, y = \begin{cases} 1, & \text{如果进行检测或者拆解次品;} \\ 0, & \text{如果不进行检测或者不拆解次品.} \end{cases} \quad (8)$$

1. 零配件 1、2 的采购与检测成本

该成本由两部分组成，第一部分采购成本为单价 a_i 与零配件采购量的乘积。为了便于计算，我们假设每轮生产中装配完成的成品数 N 恒定（无论这 N 个成品中次品有多少），对于零配件 i ，如果不进行检测，则直接进入装配，不会影响成品数，故零配件数仍为 N ；如果检测，则会剔除次品零配件，进入装配的零配件会减少。为了保证成品数恒定，所需的零配件数量需要相应增加为 $N/(1-p_i)$ 。同理计算第二部分检测成本时零配件数也需要增加，因此有

$$C_1 = \sum_{i=1}^2 \left[\frac{Na_i}{1-p_i x_i} + x_i \cdot \frac{Nc_i}{1-p_i} \right] \quad (9)$$

其中 a_i , c_i , p_i 分别表示零配件 i 的单价，检测成本，次品率。这里运用了动态规划的思想，来确定零配件的数量。

2. 成品的装配、检测成本及调换损失

$$C_2 = Nc_a + x_3 Nc_c + (1-x_3)N(1-P) \cdot (c_e + a_1 + a_2 + c_a) \quad (10)$$

其中， c_a , c_c , c_e 分别表示成品的装配成本 (Assembly Cost)，检测成本 (Checking Cost) 与调换损失 (Exchange Cost)， P 为成品的总合格率，式子的最后一项表示如果不进行检测 (检测后这一项为 0)， N 件成品中有 $1-P$ 是不合格的，需要加上调换损失以

及调换花费原材料的成本， P 的表达式如下：

$$P = \prod_{i=1}^3 [1 - p_i(1 - x_i)] \quad (11)$$

p_3 是将正品零配件装配后的产品次品率，如果在生产中没有对零配件进行检测，成品有可能由次品的零配件装配而成，那么成品的实际次品率 $1 - P$ 会超过表格给出的成品次品率 p_3 。

3. 成品的拆解成本

$$C_3 = yN(1 - P)c_d \quad (12)$$

其中 c_d 为成品的拆解费用 (Disassembly Cost)， y 的取值，即是否拆解将在 5 中进行讨论。

4. 成品的销售额

成品的销售额为售价与合格成品数量的乘积，考虑到经过检测的零配件的不合格率会将为 0，我们把零配件 1,2 的**不合格率**分别修正为 $(1 - x_1)p_1$ 和 $(1 - x_2)p_2$ ，故合格成品数量为修正后的成品的**合格率**与成品总量 N 的乘积，销售额为

$$S_1 = sN(1 - p_3) \prod_{i=1}^2 [1 - (1 - x_i)p_i] \quad (13)$$

5. 回收零配件的收益

按照题目要求，如果对检测出的不合格成品进行拆解，就需要对拆解后得到的零配件进行重新装配。根据假设，我们至多只考虑拆解一次。决定是否拆解不合格成品的决策变量 y 的取值将采用**贝叶斯公式**决策。

假设成品不合格为事件 B ，零配件 1、2 不合格分别为事件 A_1 、 A_2 ，零配件 1、2 合格的条件下成品仍不合格为事件 A_3 ，拆解一个不合格成品得到的零配件总价值为 Y ，可得：

$$P(B) = P(A_1 \cup A_2 \cup A_3) = 1 - \prod_{i=1}^3 (1 - p_i) \quad (14)$$

由贝叶斯公式 [2]

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

可以得到：

$$P(A_1|B) = \frac{P(A_1)P(B|A_1)}{P(B)} = \frac{p_1 \cdot 1}{1 - \prod_{i=1}^3 (1 - p_i)} \quad (15)$$

$$P(A_2|B) = \frac{P(A_2)P(B|A_2)}{P(B)} = \frac{p_2 \cdot 1}{1 - \prod_{i=1}^3 (1 - p_i)} \quad (16)$$

那么拆解一个不合格成品得到的零配件的价值期望为：

$$E(Y) = \sum_{i=1}^2 (1 - P(A_i|B)) a_i = \sum_{i=1}^2 \left(1 - \frac{p_i}{1 - \prod_{j=1}^3 (1 - p_j)} \right) a_i \quad (17)$$

当一个零配件被拆解后的价值 $E(Y)$ 与其拆解费用 c_d 满足 $E(Y) > c_d$ ，则拆解不合格成品，即 $y = 1$ ，否则 $y = 0$ 。

在现实生活中，工业产品生产线的零配件是不断补充的，在周期的生产中零配件的数量基本上保持恒定。不妨假设该产品有稳定的生产线。在每轮生产中，零配件来自两方面：供应商所供应的零配件、上轮生产拆解得到的零配件。由于生产是周期性的，可以认为上一次生产中拆解成品得到零配件的价值期望和本次生产得到的相等，因此可以用 $E(Y)$ 来代表本次生产中使用上轮拆解得到的零配件所省下的成本，用回收收益表示，如下式所示：

$$S_2 = yN(1 - P) \cdot E(Y) \quad (18)$$

且上轮拆解得到的零配件的检测费用和重新装配费用已经包含在本轮生产成本中，无需重复计算。故可以得出**总利润为**：

$$W = S_1 + S_2 - C_1 - C_2 - C_3 \quad (19)$$

6.2 基于穷举法的模型求解

问题二需要对生产过程中的各个阶段做出决策，针对实际情况，给出具体的决策方案。我们首先确定把企业生产该产品得到的总利润作为决策依据，建立相应的数学模型，该模型中一共有四个决策变量（是否检测零件 1、零件 2、成品和是否拆解不合格成品）。我们通过对所有可能的策略组合进行遍历，得到各组合的总利润，总利润最高的组合即为最佳决策方案。

不妨设定装配成的成品个数 N 初始值为 1000，运算后得到表3显示的最佳决策方案，表格中的数字 1 表示检测或拆解，数字 0 表示不检测或不拆解，总利润的单位为元。

表 3 问题二的决策方案表

	零件 1	零件 2	成品	不合格成品	总利润
情况 1	1	0	1	1	12 581.525
情况 2	1	0	1	1	2936.721
情况 3	1	0	1	1	12 581.525
情况 4	1	1	1	0	6800.00
情况 5	0	1	1	1	10 173.636
情况 6	1	0	1	0	17 224.211

七、问题三模型的建立与求解

问题 3 相对问题 2 由原来的 1 道工序、2 个零配件推广到 m 道工序、 n 个零配件。由于 m, n 不确定，难度较大，我们先按照和问题 2 类似的思路求解 2 道工序、8 个零配件的情况。

7.1 关于 2 道工序，8 个零配件决策方案设计的数学模型建立

注意到半成品 1 与半成品 2 的装配所需的零配件的装配成本及检测成本、装配成本、检测成本、次品率等参数都相同（其中零配件对应相同，即零配件 1 对应 4，2 对应 5，3 对应 6），因此在决策以获得最大利润时，这些决策变量的取值对应相等。详细证明如下：

根据假设，不同产品的生产过程是相互独立，互不干扰的。考虑半成品 1 和半成品 2 的装配，涉及到的决策变量数量相同，可以分别记作向量 λ_1 和 λ_2 ，由于相互独立，当总体的决策取得最优时，对应的子规划也能取到最优，即半成品 1,2 的决策都能取到最优，由于半成品 1,2 装配中涉及的相关参数都相同，故无论最优解是否唯一，必然存在一组决策变量的取值，使得 $\lambda_1 = \lambda_2$ 且最优。因此我们只需考虑其中一个半成品的决策变量取值，当其决策最优时，另外一个半成品也一定能有最优决策。这样可以降低决策变量的个数，简化模型。

1. 零配件的采购与检测成本

为了表示更简洁，定义

$$m_i = \frac{a_i}{1 - p_{x_{1i}}} + x_{1i} \frac{c_i}{1 - p} \quad \mathbf{m} = (m_1, m_2, \dots, m_8)$$

$$N_i = \frac{N}{1 - p_i x_{2i}} \quad \mathbf{N} = (N_1, N_1, N_1, N_2, N_2, N_2, N_3, N_3)$$

则零配件的采购与检测成本为

$$C_1 = \mathbf{m} \mathbf{N}^T \quad (20)$$

其中 a_i, c_i 表示 8 个零配件的单价和检测成本； N_i 表示半成品 i 的总量； p 代表不合格率，由于表中零配件，半成品，成品的不合格率相等，故统一用 p 表示，不设角标； p_i 表示半成品 i 的总的不合格率，类似于问题 2，有

$$p_1 = p_2 = 1 - (1 - p)^4 \quad p_3 = 1 - (1 - p)^3$$

x_{1i} 表示 8 个零配件是否检测， x_{2i} 表示 3 个半成品是否检测，取值与问题 2 相同。

2. 半成品的装配、检测成本

注意到半成品与问题 2 中成品类似，只是没有调换损失。

$$C_2 = c_a \sum_{i=1}^3 N_i + c_c \sum_{i=1}^3 x_{2i} N_i + \sum_{i=1}^3 [(1 - x_{2i})(1 - P_i) N_i c_{i\frac{1}{2}}] \quad (21)$$

其中半成品的装配成本与检测成本都相同，统一用 c_a, c_c 表示； $c_{i\frac{1}{2}}$ 表示制造半成品 i 的成本，由零部件的成本，装配成本，检测成本组成； P_i 为半成品 i 的合格率，有

$$\begin{aligned} c_{1\frac{1}{2}} &= c_{2\frac{1}{2}} = c_a + \sum_{i=1}^3 (a_i + x_{1i} c_i) \\ c_{3\frac{1}{2}} &= c_a + \sum_{i=7}^8 (a_i + x_{1i} c_i) \\ P_1 &= P_2 = \left[\prod_{i=1}^3 (1 - P(1 - x_{1i})) \right] [1 - p(1 - x_{21})] \\ P_3 &= \left[\prod_{i=7}^8 (1 - P(1 - x_{1i})) \right] [1 - P(1 - x_{23})] \end{aligned}$$

3. 成品的装配、检测成本及调换损失

$$C_3 = N c_a + x_3 N c_{c\text{成品}} + (1 - x_3)(1 - P) N \cdot \left(c_c + c_a + \sum_{i=1}^3 c_{i\frac{1}{2}} \right) \quad (22)$$

其中

$$P = [1 - p(1 - x_3)] \prod_{i=1}^3 (1 - P_i(1 - x_{2i}))$$

为成品合格率； x_3 为成品是否检测。

4. 成品与半成品拆解成本

$$C_4 = \sum_{i=1}^3 y_i N_i (1 - P_i) + y_4 N (1 - P) c_{d\text{成品}} \quad (23)$$

其中 i 取 1 到 3 对应 3 个半成品，取 4 时对应成品，取值仍与问题 2 相同。

5. 成品的销售额

$$S_1 = 1 - p \prod_{i=1}^3 [1 - (1 - x_{2i}) P_i] \quad (24)$$

6. 回收零配件及半成品的收益

根据问题 2 同理可得

$$S_2 = \sum_{i=1}^3 y_i N_i (1 - P_i) \cdot E(Y_i) + y N (1 - P) \cdot E(Y) \quad (25)$$

其中 Y_i 与 Y 分别表示半成品和成品拆解所得的总价值， $E(Y)$ 的计算与问题 2 基本相同。**总利润为：**

$$W = S_1 + S_2 - C_1 - C_2 - C_3 - C_4 \quad (26)$$

7.2 基于遗传算法求解决策方案

对于问题三中 2 道工序、8 个零配件的生产情况，我们建立了思路和问题二大致相同数学模型。但是，相比于问题二，问题三中的决策变量更多，模型更复杂，求解的难度呈指数型增长。尽管穷举的方法可以实现而且便于理解，对于状态空间特别大的情况，它需要遍历大量的状态组合，效率显著下降。所以在问题三的求解中，我们采用了能够提供有效搜索机制的遗传算法。

遗传算法（Genetic Algorithm, GA）借鉴了生物学现象包括遗传、突变、自然选择以及杂交等等，将候选解用二进制抽象表示为染色体，模拟进化过程，通过自然选择和突变产生新的种群，该种群在算法的下次迭代中成为初始种群，从而使种群向更好的解进化。遗传算法的主要原理如伪代码 Algorithm2 所示。

遗传算法的初始条件如下：

POPULATION_SIZE = 100，即初始种群包含 100 个个体。个体编码由一个长度为 11 的二进制列表表示，对应于零件是否检测、半成品是否检测、半成品是否拆解、成品是否检测和成品是否拆解的决策。个体的每个位随机选择 0 或 1。

使用 GA 算法连续求解 5 次得到的决策方案如图4所示，方案所在行的数字 1 表示检测或拆解，数字 0 表示不检测或不拆解，半成品和成品所在列中的“0+1”表示某种半成品或成品不检测但是进行拆解。其中 5 种决策方案只在零配件检测的决策步骤上存

Algorithm 2: 遗传算法优化策略求解

Input: 种群大小, 交叉概率, 变异概率, 最大代数等参数**Output:** 最优解及对应的总利润

```
1 随机生成初始种群;
2 for 每一代 do
3   计算每个个体的适应度;
4   初始化新种群;
5   while 新种群大小小于种群大小 do
6     选择两个父母进行交叉;
7     if 满足交叉概率 then
8       执行交叉操作生成两个子代;
9     else
10      保留父母作为子代;
11    for 每个子代 do
12      if 满足变异概率 then
13        执行变异操作;
14    将子代加入新种群;
15  使用新种群替换当前种群;
16 根据适应度函数找到最优个体;
17 return 最优解及其对应的总利润;
```

	零配件								半成品			成品
	1	2	3	4	5	6	7	8	1	2	3	\
方案1	1	1	1	1	1	1	0	1	0+1	0+1	0+1	0+1
方案2	1	1	1	1	1	1	0	0	0+1	0+1	0+1	0+1
方案3	1	1	1	1	1	1	1	1	0+1	0+1	0+1	0+1
方案4	1	1	0	1	1	0	1	0	0+1	0+1	0+1	0+1
方案5	1	1	1	1	1	1	1	0	0+1	0+1	0+1	0+1
附: 表格中的1表示检测或拆解, 数字0表示不检测或不拆解, 半成品和成品所在列中的“0+1”表示某种半成品或成品不检测但是进行拆解												

图4 基于遗传算法求解的5种决策方案

在差异。

这5种决策方案虽不尽相同, 但决策指标总利润却相差不大。这体现出GA算法的缺点, 即解释性差。

为了进一步明确决策方案, 我们又采用了穷举法来辅助设计决策方案, 将总利润最

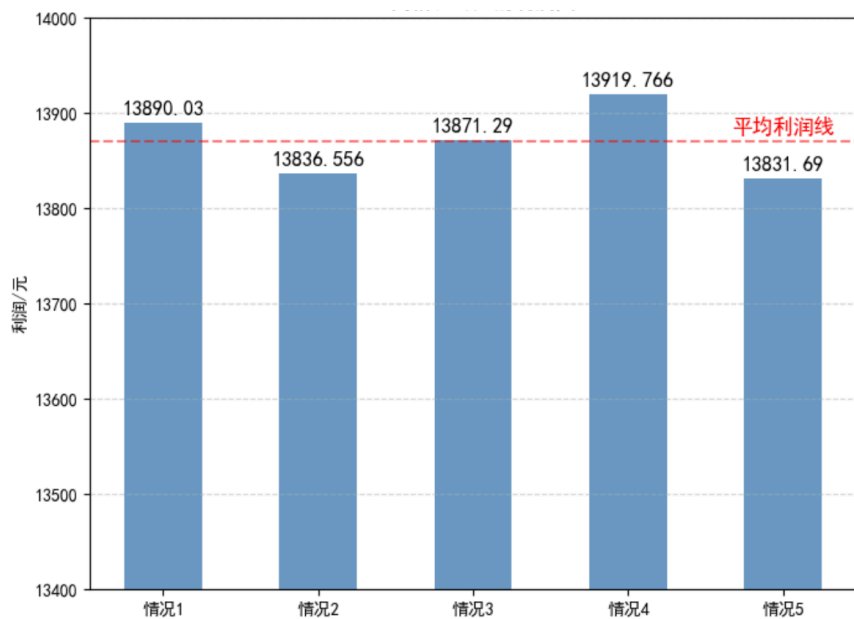


图5 基于遗传算法求解的决策方案总利润

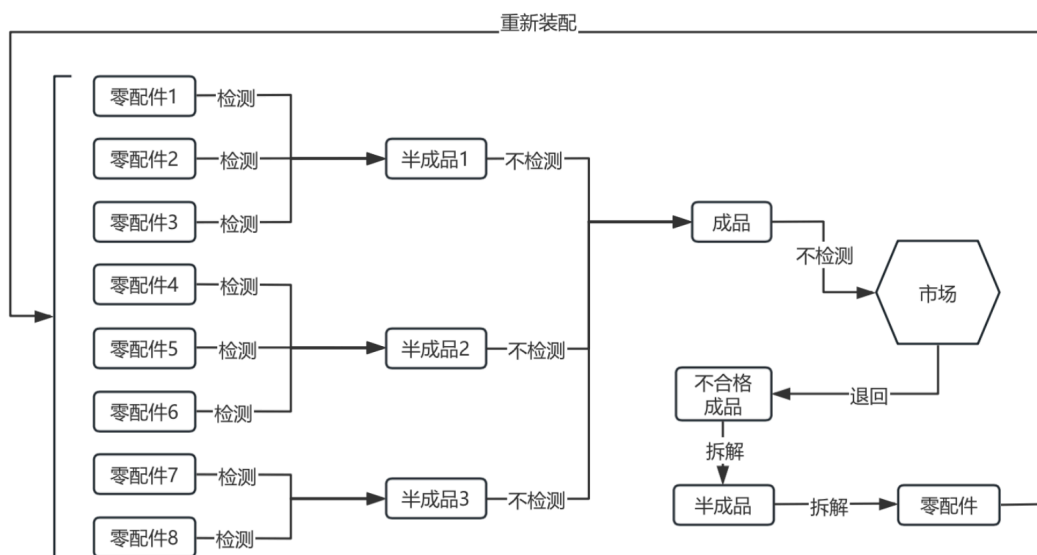


图6 问题三的决策方案流程图

高的方案确定为最终方案,即原来基于遗传算法得到的方案3。该方案的流程图如图6所示。

我们注意到,通过遗传算法求解的5种情况中,情况1和情况4的利润均比穷举法求出的利润高,这是因为在筛选种群的个体当中,部分个体存在利润为负的情况,此时将该个体的选择概率设为0,即该个体后续不会再被选中。因此,归一化后,其他个体的占比升高,总体利润随之升高。

7.3 关于 m 道工序, n 个零配件决策方案的探究

当 m, n 不确定时, 零配件、半成品个数都不确定, 装配关系也不确定, 由于不确定因素过多, 难以给出具有通用性和较强可实施性的模型。因此, 我们仅对决策方案进行探究。我们的最终目的还是最大化总利润

$$W = \sum_i S_i - \sum_j C_j \quad (27)$$

由于生产过程具有多阶段的特点, 可以考虑通过**动态规划** (Dynamic Programming, DP) 模型求解, 对于零配件, 定义每个状态为一个具体的工序检测和装配情况, 记为 $f(i, j)$, 表示第 i 道工序处理到第 j 个零配件后的最优成本, 则状态转移方程的形式为

$$f(i, j) = \min \{f(i-1, j) + c_{\text{装配}}(i, j), f(i-1, j-1) + c_{\text{检测}}(i, j), \dots\} \quad (28)$$

其中 $c_{\text{装配}}$ 和 $c_{\text{检测}}$ 分别表示装配和检测成本。根据给定的具体条件, 例如装配关系等, 可以完善这个模型, 从而进行求解。对于 m, n 较大的情况下, 由于决策变量的数目会大大增加, 动态规划的计算复杂度会变得过高, 因此可以使用启发式算法, 例如前文所提到的**模拟退火算法**和**遗传算法**进行求解。

八、问题四模型的建立与求解

8.1 基于贝叶斯估计确定次品率

问题四要求通过抽样检测方法确定零配件、半成品和成品的次品率, 问题 2 和问题 3 原先给出的次品率可以视为标称值。对于问题四, 我们采用**贝叶斯估计**和**蒙特卡洛模拟**来重新确定次品率。

贝叶斯估计是统计推断的一种方法, 主要分为如下 3 个步骤:

1. 设定先验分布: 设定各个零配件、成品次品率的先验分布, 确定我们对次品率的信念。
2. 模拟二项分布抽样: 基于贝叶斯定理, 用观测数据修正对次品率的估计, 得到新的后验分布。
3. 蒙特卡洛模拟样本量: 进一步评估后验分布的特性, 生成新的观测数据并不断更新后验分布。

我们假设 $Beta$ 分布为先验分布。因为 $Beta$ 分布是一个定义在 $[0,1]$ 区间上的分布, 适合用于次品率的不确定性, 而且 $Beta$ 分布是二项分布的共轭分布, 通过贝叶斯定理更新后, 后验分布仍然是 $Beta$ 分布。故假设 $Beta$ 分布为先验分布, 能很好地适应各种先验信息, 简化计算过程。

下面先推导证明 *Beta* 分布通过贝叶斯定理更新后，后验分布仍然是 *Beta* 分布。

如果我们有一个 *Beta* 先验分布 $Beta(\alpha, \beta)$ ，并观测到 X 次成功和 $n - X$ 次失败，观测数据的似然函数是二项分布：

$$P(X|\theta, n) = C_n^X \theta^X (1 - \theta)^{n-X}$$

使用贝叶斯定理，我们可以得到后验分布：

$$P(\theta|X, n) \propto P(X|\theta, n) \cdot P(\theta)$$

代入似然函数和先验分布的公式：

$$P(\theta|X, n) \propto \theta^X (1 - \theta)^{n-X} \cdot \theta^{\alpha-1} (1 - \theta)^{\beta-1}$$

简化得到：

$$P(\theta|X, n) \propto \theta^{\alpha+X-1} (1 - \theta)^{\beta+n-X-1}$$

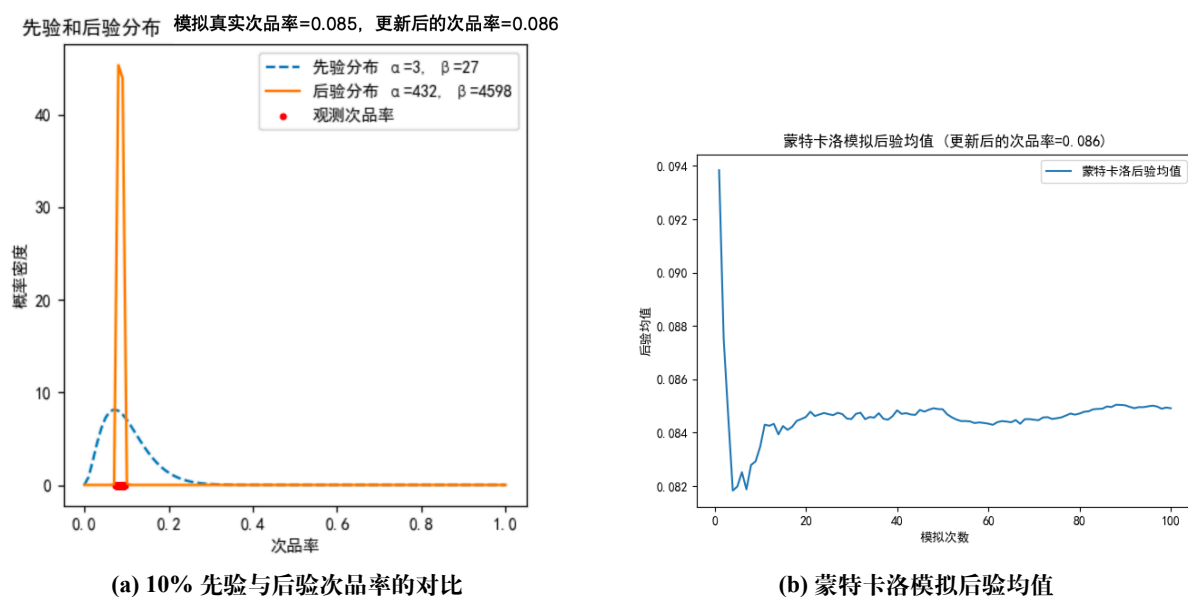


图7 10% 先验值下的蒙特卡洛模拟

假设商家给定的标称值和实际次品率较为接近，下面以 10% 标称值为例，介绍如何修正模拟真实次品率。

如图10,我们使用较小数值的 α, β 完成弱信息先验,这里设定先验分布为 $Beta(3,27)$ 。不妨假设真实次品率落在以 10% 为中心的区间上，区间的长度可以根据我们对先验概率的相信程度进行调整。这里我们假设真实次品率为 8%~12%，以 0.5% 作为步长。以假设的真实次品率为 0.085 为例，通过贝叶斯估计和蒙特卡洛模拟之后，我们得到后验分布的参数为 $\theta \sim Beta(435,4625)$ ，更新后的次品率为 0.086。

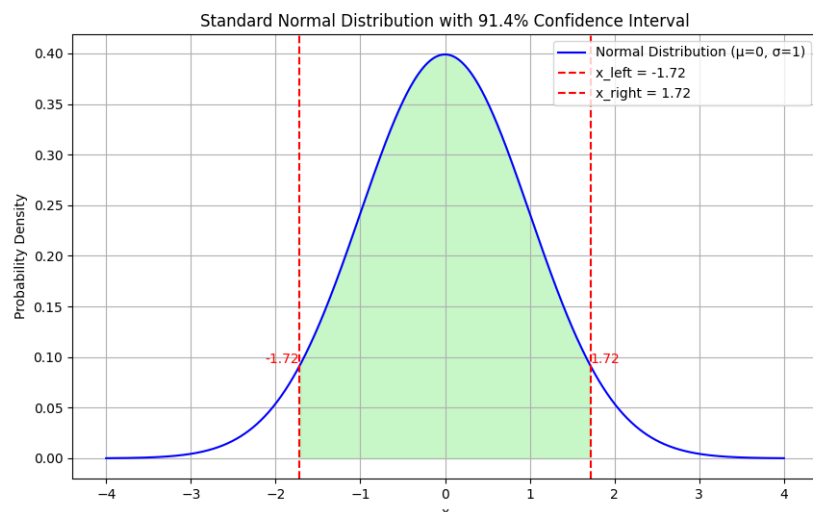


图 8 正态分布图

通过观察图10(b)，可以认为当模拟次数达到 20 时，后验均值逐渐收敛。逐渐增大模拟次数，后验均值基本稳定在 0.084 至 0.086 之间。显然相比之前假设的真实次品率，更新后的次品率更接近于真实值。此处我们取更新后的次品率为 8.6%。

如图8所示，绘制 1000 个以 0 为均值，以 1 为标准差的正态分布的点。由 8.6% 的次品率估计值可知，绿色部分占比为 91.4%，此时可以计算出两侧 x 的值为 -1.72 和 1.72。将绿色部分定义为正品，空白部分定义为次品，进行一次抽样，抽样数目选择问题一中 95% 的信度计算出的抽样数目。以抽出次品的个数与抽样数目的比值作为抽样检测方法得到的次品率。这里我们模拟数据，选择抽样 43 次，得到 4 个次品，此时可计算出次品率为 9.3%。

同理可计算出标称值为 5% 和 20% 的抽样模拟次品率。

8.2 重新求解问题二和问题三的决策方案

通过贝叶斯估计，我们成功确定次品率标称值为 5%、10%、20% 的产品批次通过抽样检测得到的次品率分别为 4.0%、9.3% 和 18.6%。将抽样检测得到的次品率代入问题二、问题三建立的数学模型中，得到最佳方案和相应的决策指标。表4中的数字 1、0 的含义与表3相同，1 表示检测或拆解，0 表示不检测或不拆解。

图9展示的是次品率更改前后的利润对比图，由此可知，当次品率降低，选择最优决策方案后，成品的总利润增加。

由于问题三的生产情况中零配件、半成品和成品的次品率均为 10%，所以重新求解的最优决策方案与问题三相同，如图4所示，即零配件均需检测，半成品和成品均不检测，对于不合格的半成品和成品均进行拆解，总利润为 13860.40 元。

当取不同的次品率计算问题三时，可以得到不同的单件成品的利润。利润不包含回

表 4 重做问题二的决策方案表

	零件 1	零件 2	成品	不合格成品	总利润
情况 1	1	0	1	1	13 284.58
情况 2	1	0	1	1	4244.10
情况 3	1	0	1	1	13 284.58
情况 4	1	1	1	0	8099.97
情况 5	0	1	1	1	11 260.61
情况 6	1	0	1	0	18 545.22

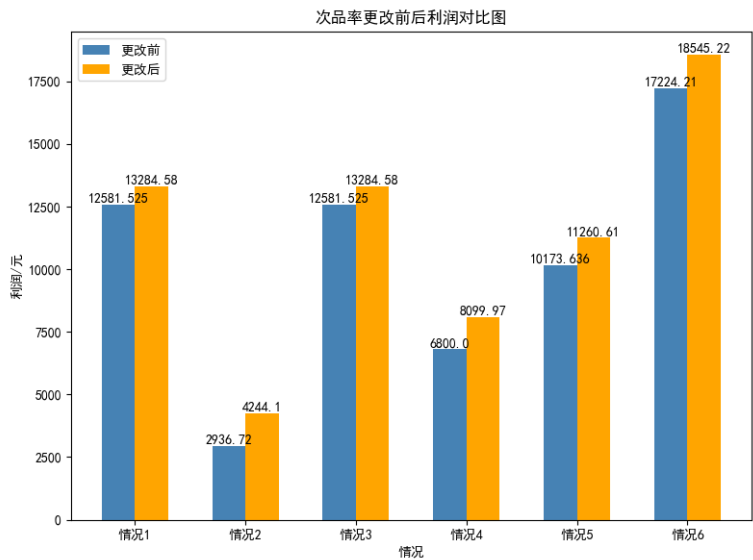


图 9 次品率更改前后利润对比图

收收益时，如图10(a) 所示，利润包含回收收益时，如图10(b) 所示。从图10(a) 中可以看出，随着次品率升高，单件成品利润逐渐降低，同时各个点可以被线性拟合，且相关系数为 1，拟合度很好。从图10(b) 中可以看出，随着次品率升高，单件成品利润逐渐升高，这是因为次品率升高后，成品为次品的概率升高，回收收益显著增长。各个点可以被线性拟合，且相关系数为 0.98，拟合度良好。

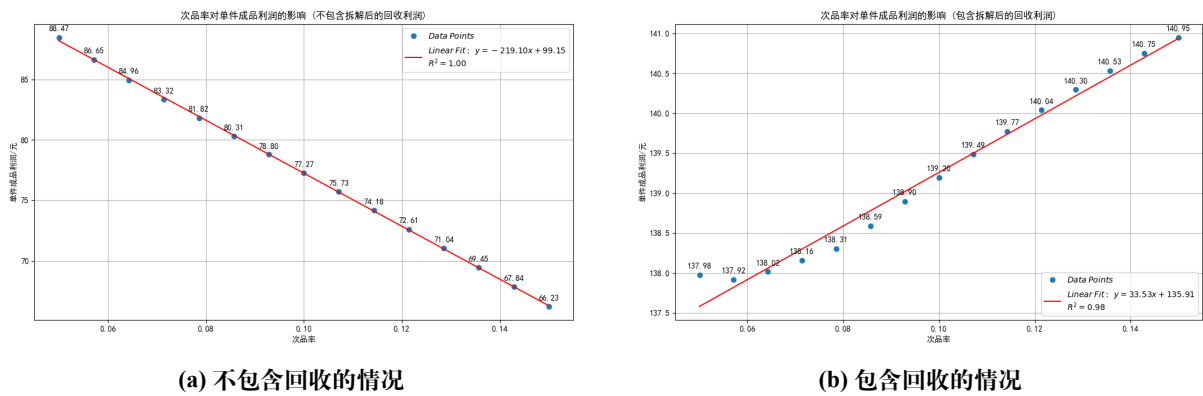


图 10 单个成品利润和次品率的关系

九、灵敏度分析

9.1 问题一灵敏度分析

问题一使用了模拟退火算法，需要关注初始温度、冷却速率与终止温度，通过逐步修改这些参数进行实验，我们设置了较高的初始温度、较慢的冷却速率和较低的终止温度，虽然增加了计算时间，但由于要求解的模型本身并不是很复杂，因此该设置有利于提升计算精度和鲁棒性。

9.2 问题二灵敏度分析

问题二基于穷举法，未引入超参数，所有参数基于原始数据，无需进行灵敏度分析。

9.3 问题三灵敏度分析

问题三求解时使用了遗传算法，需关注种群大小、交叉概率与变异概率，实验中，我们设置较大的种群、较高的交叉概率与一定的变异概率，同样通过增加计算时间的方法来提高模型的准确度和鲁棒性。

9.4 问题四灵敏度分析

问题四使用了贝叶斯估计和蒙特卡洛模拟。

贝叶斯估计中基于次品率的先验分布为 $Beta(\alpha, \beta)$ ，假设次品率为 10%，则使用弱信息先验可认为 $10\alpha = \beta$ ，而 α 与 β 具体值的选取应基于对商家的信任程度（若次品率为商家标称值）或样本容量（若次品率来自上一次抽样检测），应根据实际情况选取。

蒙特卡洛模拟中需要确定合适的概率分布以及随机样本的生成数。在概率分布的选择上，应当根据变量符合的条件选择合适的概率分布，例如正态分布、均匀分布、泊松分布，在本题中采用二项分布抽取数据。同时，随机样本的生成数应当足够大，以保证模拟结果的可靠性。

十、模型的评价

10.1 模型的优点

- 简化复杂性：注意到半成品 1 和半成品 2 完全等价，将 16 个决策变量简化为 11 个，降低了计算复杂性。
- 优化资源利用：启发式算法可以帮助确定最佳策略，以最有效地利用可用资源。
- 可靠性强：问题二和问题三的模型可以用穷举法求解，参数来源于原始数据，更加可靠。

- 充分结合实际：一些近似处理在数学上或者生产实际中具有合理性，例如把超几何分布近似为二项分布，以及对回收零配件的处理。

10.2 模型的缺点

- 模型基于稳定的生产线，未考虑市场行情、经济形势等外部因素带来的生产线调整，存在局限性。
- 模型对于工序、零配件个数及组装情况有明确要求，难以对其变化作出相应调整。

10.3 模型的意义和展望

本模型对生产的各个过程进行了决策分析，旨在降低成本并增加利润，同时提出了更加合理的抽样检测方法，使得通过抽样得来的次品率更加接近于真实值，从而让决策更加准确可靠。可以考虑结合外部数据，例如该生产线的产量随时间的变化、该产品在市场需求量等，适当调整不同生产批次中总量 N 的值，使得模型更加贴合实际。

参考文献

- [1] 卫淑之, 熊德文, 大学数学概率论与数理统计：基于案例分析, 北京：高等教育出版社, 2020.
- [2] 武爱文, 冯卫国, 概率论与数理统计, 上海：上海交通大学出版社, 2014.
- [3] Grant, E. L. 质量统计控制方法, 北京：机械工业出版社, 1999.1.
- [4] 司守奎, 孙玺菁 *Python* 数学建模算法与应用, 北京：国防工业出版社, 2023.2.

附件

A 详细代码

1.1 作图代码

Listing 1: 二项式累计概率计算

```
import matplotlib.pyplot as plt
from scipy.special import comb

def cumulative_binomial_prob(n, d, p=0.1):
    """计算从0到d的二项式累积概率"""
```

```

    return sum(comb(n, k) * p**k * (1-p)**(n-k) for k in range(d+1))

def find_d_for_threshold(n, threshold=0.95):
    """找到使得累积概率首次超过或等于阈值的d"""
    for d in range(n + 1):
        if cumulative_binomial_prob(n, d) >= threshold:
            return d
    return None

# 生成n的范围
n_values = range(1, 200)

# 找到对应的d值
d_values = [find_d_for_threshold(n) for n in n_values]

# 剔除因为n太小而找不到d的情况
filtered_pairs = [(n, d) for n, d in zip(n_values, d_values) if d is not None]

# 提取n和d的值
n_points, d_points = zip(*filtered_pairs)

# 绘制散点图
# 更改点的大小
plt.scatter(n_points, d_points, label='Data Points', s = 2)

# 添加图例和标题
plt.xlabel('n')
plt.ylabel('d')
plt.title('Scatter Plot of (n, d) Pairs when p = 0.1')

annotate_points = [(n_points[29], d_points[29]),
                    (n_points[89], d_points[89]),
                    (n_points[149], d_points[149])]

for x, y in annotate_points:
    plt.annotate(f'({x}, {y})', # 标注的文本
                 xy=(x, y),    # 箭头指向的点
                 xytext=(30, -30), # 文本相对于箭头指向点的位置
                 textcoords='offset points', # 文本坐标是相对于xy指定的点
                 arrowprops=dict(facecolor='red', shrink=0.05)) # 箭头属性

plt.legend()
plt.show()

```

Listing 2: 问题三遗传算法多次求解利润图

```

# 条形图
import matplotlib.pyplot as plt

```

```

import numpy as np

# 数据
x = np.array(['情况1', '情况2', '情况3', '情况4', '情况5'])
y = np.array([13890.03, 13836.556, 13871.29, 13919.766, 13831.69])

# 条形图
plt.figure(figsize=(8, 6))

plt.rcParams['font.sans-serif'] = ['SimHei'] # 用来正常显示中文标签
plt.rcParams['axes.unicode_minus'] = False # 用来正常显示负号
plt.bar(x, y, color='steelblue', alpha=0.8, width=0.5)

# 设置坐标轴
# 设置Y轴起始值
plt.ylim(13400, 14000)
# 加上网格线
plt.grid(axis='y', linestyle='--', alpha=0.5)

for i in range(len(x)):
    plt.text(i, y[i]+5, y[i], ha='center', va='bottom', fontsize=12)

plt.ylabel('利润/元')
plt.title('不同情况对应的利润图')

# 添加平均值
plt.axhline(y=13869.866, color='r', linestyle='--', alpha=0.5)
plt.text(4, 13869.866+5, '平均利润线', ha='center', va='bottom', fontsize=12, color='r')

plt.show()

```

Listing 3: 次品率灵敏度分析

```

import itertools
import numpy as np
from functools import reduce
import operator
import matplotlib.pyplot as plt
from scipy.stats import linregress

# 参数定义
N = 100 # 生产量
sell_price = 200 # 成品卖出价格
per_assembly_cost = 8 # 装配成本
per_semi_final_disassembly_cost = 6 # 半成品拆解成本

```



```

per_final_disassembly_cost = 10 # 成品拆解成本
per_replacement_cost = 40 # 调换损失

# 零配件1至8 半成品1至3 成品的次品率
p_bad = 0.086

# 零配件1至8的购买单价
parts_prices = [2, 8, 12, 2, 8, 12, 8, 12]

# 零配件1至8的检测成本
parts_testing_costs = [1, 1, 2, 1, 1, 2, 1, 2]

# 半成品和成品的检测成本
semi_final_testing_cost = 4
final_testing_cost = 6

# 利润计算函数
def calculate_profit(N, parts_prices, parts_testing_costs, semi_final_testing_cost,
                    final_testing_cost,
                    per_assembly_cost, per_semi_final_disassembly_cost,
                    per_final_disassembly_cost,
                    per_replacement_cost, sell_price, p_bad, detect_parts,
                    detect_semi_finals, disassemble_semi_finals, detect_final,
                    disassemble_final):

    # 初始化利润
    profit = 0

    # 计算半成品个数
    p_final_1 = p_final_2 = 1 - (1 - (1 - p_bad) ** 4) * detect_semi_finals[0]
    p_final_3 = 1 - (1 - (1 - p_bad) ** 3) * detect_semi_finals[1]

    num_semi_1 = num_semi_2 = N / p_final_1
    num_semi_3 = N / p_final_3

    # 计算零配件个数
    num_1 = num_4 = num_semi_1 / (1 - p_bad * detect_parts[0])
    num_2 = num_5 = num_semi_1 / (1 - p_bad * detect_parts[1])
    num_3 = num_6 = num_semi_1 / (1 - p_bad * detect_parts[2])
    num_7 = num_semi_3 / (1 - p_bad * detect_parts[3])
    num_8 = num_semi_3 / (1 - p_bad * detect_parts[4])

    # 成品回收均值
    E_revenue_final = 2 * (1 - (1 - p_bad) ** 4) / (1 - (1 - p_bad) ** 12) * (60 +
        detect_parts[0] * parts_testing_costs[0]) + \
        detect_parts[1] * parts_testing_costs[1] + detect_parts[2] *
        parts_testing_costs[2] + \

```

```

        (1 - (1 - p_bad) ** 3) / (1 - (1 - p_bad) ** 12) * (28 + detect_parts[3] *
            parts_testing_costs[3] + detect_parts[4] * parts_testing_costs[4])

# 半成品1和2的回收均值
E_revenue_semi_1 = E_revenue_semi_2 = (p_bad / (1 - (1 - p_bad) ** 3) * (1 - p_bad)) * (2 +
    8 + 12)
E_revenue_semi_3 = (p_bad / (1 - (1 - p_bad) * (1 - p_bad)) * (1 - p_bad)) * (8 + 12)

# 零配件采购总成本
total_parts_cost = sum(num * price for num, price in zip([num_1, num_2, num_3, num_4,
    num_5, num_6, num_7, num_8], parts_prices))

# 零配件检测总成本
total_parts_testing_cost = 2 / (1 - p_bad) * num_semi_1 * sum(detect_parts[i] *
    parts_testing_costs[i] for i in range(3)) + \
    1 / (1 - p_bad) * num_semi_3 * sum(detect_parts[i] *
    parts_testing_costs[i] for i in range(3, 5))

# 单个半成品1和2的成本
semi_prices_1 = semi_prices_2 = per_assembly_cost + parts_prices[0] + parts_prices[1] +
    parts_prices[2] + \
        sum(detect_parts[i] * parts_testing_costs[i] for i in range(3))
semi_prices_3 = per_assembly_cost + parts_prices[6] + parts_prices[7] + detect_parts[3] *
    parts_testing_costs[3] + \
        detect_parts[4] * parts_testing_costs[4]

# 半成品合格率
P1 = P2 = (1 - p_bad * (1 - detect_semi_finals[0])) * reduce(operator.mul, ((1 - p_bad * (1
    - detect_parts[i])) for i in range(3)))
P3 = (1 - p_bad * (1 - detect_parts[3])) * (1 - p_bad * (1 - detect_parts[4])) * (1 - p_bad
    * (1 - detect_semi_finals[1]))

# 半成品的装配+检测+替换成本
total_semi = per_assembly_cost * (2 * num_semi_1 + num_semi_3) + \
    semi_final_testing_cost * (2 * num_semi_1 * detect_semi_finals[0] + num_semi_3 *
    detect_semi_finals[1]) + \
    2 * (1 - detect_semi_finals[0]) * num_semi_1 * (1 - P1) * semi_prices_1 + \
    (1 - detect_semi_finals[1]) * num_semi_3 * (1 - P3) * semi_prices_3

# 成品合格率
final_P = (1 - P1 * (1 - detect_semi_finals[0])) * (1 - P1 * (1 - detect_semi_finals[0])) *
    (1 - P3 * (1 - detect_semi_finals[1])) * (1 - p_bad * (1 - detect_final))

# 成品检测成本
total_final_testing_cost = N * detect_final * final_testing_cost

# 成品装配成本

```

```

total_assembly_cost = N * per_assembly_cost

# 成品调换损失
replacement_cost = (per_replacement_cost + 2 * semi_prices_1 + semi_prices_3 +
                    per_assembly_cost) * final_P * N * (1 - detect_final)

# 成品的总成本
total_final = total_final_testing_cost + total_assembly_cost + replacement_cost

# 拆解成本
disassembly_cost = 2 * disassemble_semi_finals[0] * num_semi_1 * (1 - P1) *
                  per_semi_final_disassembly_cost + \
                  disassemble_semi_finals[1] * num_semi_3 * (1 - P3) *
                  per_semi_final_disassembly_cost + \
                  disassemble_final * N * (1 - final_P) * per_final_disassembly_cost

# 销售额
revenue = sell_price * N * (1 - p_bad * (1-(1-detect_semi_finals[0] * P1)) *
                          (1-(1-detect_semi_finals[0] * P1)) * (1-(1-detect_semi_finals[1]) * P3))

# 回收利润
recycle = E_revenue_final * N * disassemble_final * (1 - final_P) + \
          E_revenue_semi_1 * num_semi_1 * disassemble_semi_finals[0] * (1 - P1) * 2 + \
          E_revenue_semi_3 * num_semi_3 * disassemble_semi_finals[1] * (1 - P3)

# 总利润计算
profit = revenue + recycle - total_parts_testing_cost - total_parts_cost - total_semi -
        total_final - disassembly_cost

return profit

# 动态规划表
states = list(itertools.product([0, 1], repeat=11)) # 5个零件检测决策 + 2个半成品检测决策 +
            2个半成品拆解决策 + 1个成品检测决策 + 1个成品拆解决策

ANSWER = np.zeros(len(states))

p_bad_profits = []
p_bad_range = np.linspace(0.05, 0.15, 15)

for p_bad in p_bad_range:
    # 动态规划计算
    for idx, state in enumerate(states):
        detect_parts = state[:5] # 零件12378是否检测
        detect_semi_finals = state[5:7] # 半成品13是否检测
        disassemble_semi_finals = state[7:9] # 是否拆解半成品13
        detect_final, disassemble_final = state[9:11] # 成品是否检测, 是否拆解

```

```

ANSWER[idx] = calculate_profit(
    N, parts_prices, parts_testing_costs, semi_final_testing_cost, final_testing_cost,
    per_assembly_cost, per_semi_final_disassembly_cost, per_final_disassembly_cost,
    per_replacement_cost, sell_price, p_bad, detect_parts,
    detect_semi_finals, disassemble_semi_finals, detect_final, disassemble_final
)

# 找最优策略
optimal_strategy = max(states, key=lambda state: ANSWER[list(states).index(state)])

print("最优策略: ")
for i, decision in enumerate(optimal_strategy):
    print(f"第{i+1}个决策 (0为不检测/不拆解, 1为检测/拆解) : {decision}")

# 计算总利润
optimal_profit = ANSWER[list(states).index(optimal_strategy)]
print(f"最优策略下的总利润: {optimal_profit}")
optimal_profit = round(optimal_profit / 100, 3)
p_bad_profits.append(optimal_profit)

# 绘制灵敏度分析图
plt.figure(figsize=(10, 6))

plt.rcParams['font.sans-serif'] = ['SimHei']

plt.rcParams['axes.unicode_minus'] = False

plt.xlabel("次品率")
# plt.xlabel("卖价/元")
# plt.xlabel("零件检测成本/元")
plt.ylabel("单件成品利润/元")

# 计算线性回归
slope, intercept, r_value, p_value, std_err = linregress(p_bad_range, p_bad_profits)

# 根据线性回归的参数计算拟合线
fit_line = slope * p_bad_range + intercept

# 绘制原始数据点
plt.plot(p_bad_range, p_bad_profits, marker='o', linestyle='', label='$Data\_Points$')

# 绘制拟合线
plt.plot(p_bad_range, fit_line, color='red', label=f'$Linear\_Fit: \$
    $y={slope:.2f}x+{intercept:.2f}$\n$R^2={r_value**2:.2f}$')

# 添加文本标签

```

```

for i, (p_bad, p_bad_profit) in enumerate(zip(p_bad_range, p_bad_profits)):
    plt.text(p_bad, p_bad_profit + 0.08, f"{p_bad_profit:.2f}", ha='center')

# plt.title("次品率对单件成品利润的影响（不包含拆解后的回收利润）")
plt.title("次品率对单件成品利润的影响（包含拆解后的回收利润）")
# plt.title("卖价对单件成品利润的影响")
# plt.title("零件检测成本对单件成品利润的影响")

# 添加图例
# plt.legend(loc='upper right')
plt.legend(loc='lower right')

# 显示图表
plt.grid(True)

plt.tight_layout()
plt.show()

```

Listing 4: 正态分布

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

# 生成正态分布曲线
mu = 0 # 均值
sigma = 1 # 标准差

# 生成x轴数据点
x = np.linspace(mu - 4*sigma, mu + 4*sigma, 1000)

# 计算正态分布的概率密度函数
y = norm.pdf(x, mu, sigma)

# 计算满足中间部分概率为91.4%的两侧x值
confidence_level = 0.914
z_score = norm.ppf((1 + confidence_level) / 2)

# 计算x值
x_left = mu - z_score * sigma
x_right = mu + z_score * sigma

# 随机抽取43个样本
samples = norm.rvs(size=43, loc=mu, scale=sigma)

# 统计有几个样本落在两侧
count_left = np.sum(samples < x_left)

```

```

count_right = np.sum(samples > x_right)

print((count_left + count_right) / 43)

# 绘制正态分布曲线
plt.figure(figsize=(10, 6))
plt.plot(x, y, label='Normal Distribution ( =0, =1)', color='blue')

# 标注中间部分
plt.fill_between(x, y, where=(x >= x_left) & (x <= x_right), color='lightgreen', alpha=0.5)

# 标注x值
plt.axvline(x=x_left, color='red', linestyle='--', label=f'x_left = {x_left:.2f}')
plt.axvline(x=x_right, color='red', linestyle='--', label=f'x_right = {x_right:.2f}')

# 文本标注
plt.text(x_left, norm.pdf(x_left, mu, sigma), f'{x_left:.2f}', fontsize=10,
        verticalalignment='bottom', horizontalalignment='right', color='red')
plt.text(x_right, norm.pdf(x_right, mu, sigma), f'{x_right:.2f}', fontsize=10,
        verticalalignment='bottom', horizontalalignment='left', color='red')

# 添加标题和标签
plt.title('Standard Normal Distribution with 91.4% Confidence Interval')
plt.xlabel('x')
plt.ylabel('Probability Density')
plt.legend()

# 显示图形
plt.grid(True)
plt.show()

```

Listing 5: 问题四——重做问题二——利润对比复式条形图

```

import numpy as np
import matplotlib.pyplot as plt

# 数据
x_labels = ['情况1', '情况2', '情况3', '情况4', '情况5', '情况6']
data1 = np.array([12581.525, 2936.72, 12581.525, 6800, 10173.636, 17224.21]) # 第一组数据
data2 = np.array([13284.58, 4244.1, 13284.58, 8099.97, 11260.61, 18545.22]) # 第二组数据

# 生成条形图
fig, ax = plt.subplots(figsize=(8, 6))

# 设置x轴的位置
x = np.arange(len(x_labels))

```

```

# 设置条形图宽度
bar_width = 0.3

# 绘制条形图，每组数据并排显示
bar1 = ax.bar(x - bar_width/2, data1, bar_width, label='更改前', color='steelblue')
bar2 = ax.bar(x + bar_width/2, data2, bar_width, label='更改后', color='orange')

# 添加文字说明
for i in range(len(x)):
    ax.text(x[i] - bar_width/2, data1[i] + 1, f'{data1[i]}', ha='center', va='bottom')
    ax.text(x[i] + bar_width/2, data2[i] + 1, f'{data2[i]}', ha='center', va='bottom')

# 设置标签
plt.rcParams['font.sans-serif'] = ['SimHei'] # 用来正常显示中文标签
plt.rcParams['axes.unicode_minus'] = False # 用来正常显示负号
ax.set_xlabel('情况')
ax.set_ylabel('利润/元')
ax.set_title('次品率更改前后利润对比图')
ax.set_xticks(x)
ax.set_xticklabels(x_labels)
ax.legend()

plt.tight_layout()
plt.show()

```

1.2 问题一代码

Listing 6: 模拟退火

```

import numpy as np
from scipy.special import comb # 用于计算组合数
import random

# 目标函数
def objective(n, d):
    # 累积和，表示公式的右边部分
    cumulative_prob = 0
    for i in range(0, d + 1):
        cumulative_prob += comb(n, i) * (0.1 ** i) * (0.9 ** (n - i))

    # 我们要最小化 目标值和误差
    return abs(cumulative_prob - 0.1)

```

```

# 模拟退火算法
def simulated_annealing(init_temp, min_temp, cooling_rate, max_iter):
    # 随机初始化n和d, n要大于10
    n = random.randint(11, 100)
    d = random.randint(0, n // 10)

    # 当前温度
    current_temp = init_temp

    # 当前解及其目标函数值
    current_sol = (n, d)
    current_cost = objective(n, d)

    # 记录最优解
    best_sol = current_sol
    best_cost = current_cost

    for _ in range(max_iter):
        if current_temp < min_temp:
            break

        # 邻域内生成新解
        new_n = max(11, min(100, current_sol[0] + random.randint(-5, 5))) # 保证n至少为11
        new_d = max(0, min(new_n // 10, current_sol[1] + random.randint(-1, 1))) # d不能超过n//10
        new_sol = (new_n, new_d)

        # 计算新解的目标值
        new_cost = objective(new_n, new_d)

        # 判断是否接受新解
        if new_cost < current_cost:
            current_sol = new_sol
            current_cost = new_cost
        else:
            # 以一定概率接受较差解 (模拟退火中的“退火”过程)
            if np.exp(-(new_cost - current_cost) / current_temp) > random.random():
                current_sol = new_sol
                current_cost = new_cost

        # 更新最优解
        if current_cost < best_cost:
            best_sol = current_sol
            best_cost = current_cost

        # 降低温度
        current_temp *= cooling_rate

```



```

    return best_sol, best_cost

# 运行模拟退火算法100次
init_temp = 1000
min_temp = 1
cooling_rate = 0.95
max_iter = 1000
num_runs = 1000

# 存储每次的结果
results = []

for _ in range(num_runs):
    best_sol, best_cost = simulated_annealing(init_temp, min_temp, cooling_rate, max_iter)
    results.append((best_sol, best_cost))

# 提取所有n和d的值
ns = [sol[0] for sol, cost in results]
ds = [sol[1] for sol, cost in results]

# 计算n和d的平均值
avg_n = np.mean(ns)
avg_d = np.mean(ds)

# 计算平均误差
average_error = np.mean([cost for sol, cost in results])

# 输出结果
print(f"运行了 {num_runs} 次模拟退火算法的结果:")
print(f"n的平均值: {avg_n:.2f}")
print(f"d的平均值: {avg_d:.2f}")
print(f"平均误差: {average_error:.4f}")

# 输出每次的最佳解
for i, (best_sol, best_cost) in enumerate(results):
    print(f"第 {i + 1} 次: n = {best_sol[0]}, d = {best_sol[1]}, 误差 = {best_cost:.4f}")

```

1.3 问题二代码

```

import numpy as np

# 参数定义
N = 1000 # 生产量
part1_per_price = 4 # 零配件1单位成本

```

```

part2_per_price = 18 # 零配件2单位成本
sell_price = 56 # 卖出价格
per_assembly_cost = 6 # 装配成本

def profit_earned(N, p1, p2, p3, part1_per_price, part2_per_price,
                  per_part1_testing_cost, per_part2_testing_cost, per_final_testing_cost,
                  per_replacement_cost, per_scrap_cost, per_assembly_cost,
                  detect_part1, detect_part2, detect_final, disassemble):

    # 零配件检测和采购
    part1_quality_rate = (1 - p1 * detect_part1) # 筛选后零配件1占原总数的比值
    part2_quality_rate = (1 - p2 * detect_part2) # 筛选后零配件2占原总数的比值

    part1_quantity = N / part1_quality_rate # 零配件1采购数量
    part2_quantity = N / part2_quality_rate # 零配件2采购数量

    part1_price = part1_quantity * part1_per_price # 零配件1采购成本
    part2_price = part2_quantity * part2_per_price # 零配件2采购成本

    part1_testing_cost = detect_part1 * (N / (1 - p1)) * per_part1_testing_cost # 零配件1检测成本
    part2_testing_cost = detect_part2 * (N / (1 - p2)) * per_part2_testing_cost # 零配件2检测成本

    # 成品装配成本
    assembly_cost = N * per_assembly_cost # 装配成本

    # 成品检测成本
    final_testing_cost = detect_final * N * per_final_testing_cost

    # 成品合格率
    product_quality_rate = (1 - p1 * (1 - detect_part1)) * (1 - p2 * (1 - detect_part2)) * (1 -
        p3 * (1 - detect_final))

    # 调换损失
    replacement_cost = (1 - detect_final) * N * (1 - product_quality_rate) *
        (per_replacement_cost + part1_per_price + part2_per_price + per_assembly_cost)

    # 拆解成本和回收收益
    scrap_cost = disassemble * N * (1 - product_quality_rate) * per_scrap_cost

    E_revenue = part1_per_price * (1 - p1 / (1 - (1 - p1) * (1 - p2) * (1 - p3))) + \
        part2_per_price * (1 - p2 / (1 - (1 - p1) * (1 - p2) * (1 - p3))) # 回收收益
    scrap_revenue = disassemble * N * (1 - product_quality_rate) * E_revenue

    # 卖出的收益
    P3 = p3
    P2 = p2 * (1 - detect_part2)

```

```

P1 = p1 * (1 - detect_part1)
sell_quality_rate = 1 - (P1 + P2 + P3 - P1 * P2 - P1 * P3 - P2 * P3 + P1 * P2 * P3)
selling_revenue = sell_quality_rate * N * sell_price

# 总利润计算
profit = (part1_price + part2_price + part1_testing_cost + part2_testing_cost + \
          assembly_cost + final_testing_cost + replacement_cost + scrap_cost -
          scrap_revenue - selling_revenue) * (-1)

return profit, scrap_revenue

# 动态规划表
# d1表示是否检测零配件1, d2表示是否检测零配件2, d3表示是否检测成品, d4表示是否拆解不合格成品
# 1表示拆或者检测, 0表示不拆/不检测

answer = [(d1, d2, d3, d4)
           for d1 in [0, 1] for d2 in [0, 1] for d3 in [0, 1] for d4 in [0, 1]]

ANSWER = np.zeros((2, 2, 2, 2))
minus = np.zeros((2, 2, 2, 2))

# 参数
p1 = 0.1 # 零配件1次品率
per_part1_testing_cost = 2 # 零配件1检测成本
p2 = 0.1 # 零配件2次品率
per_part2_testing_cost = 3 # 零配件2检测成本
p3 = 0.1 # 成品次品率
per_final_testing_cost = 3 # 成品检测成本
per_replacement_cost = 6 # 调换损失
per_scrap_cost = 5 # 拆解成本

# 动态规划dp
for d1 in [0, 1]:
    for d2 in [0, 1]:
        for d3 in [0, 1]:
            for d4 in [0, 1]:
                ANSWER[d1, d2, d3, d4], minus[d1, d2, d3, d4] = profit_earned(N, p1, p2, p3,
                                     part1_per_price, part2_per_price,
                                     per_part1_testing_cost, per_part2_testing_cost,
                                     per_final_testing_cost,
                                     per_replacement_cost, per_scrap_cost,
                                     per_assembly_cost, d1, d2, d3, d4)

# 找最优策略
optimal_strategy = max(answer, key=lambda state: ANSWER[state])

print("最优策略: ")

```

```

print(f"零件1是否检测, 1为检测, 0为不检测: {optimal_strategy[0]}")
print(f"零件2是否检测, 1为检测, 0为不检测: {optimal_strategy[1]}")
print(f"成品是否检测, 1为检测, 0为不检测: {optimal_strategy[2]}")
print(f"是否拆解不合格成品, 1为拆解, 0为不拆解: {optimal_strategy[3]}")

# 计算最大利润
optimal_profit = ANSWER[optimal_strategy]
print(f"最优策略下的总利润: {optimal_profit}")

# 计算回收的收益
optimal_minus = minus[optimal_strategy]
print(f"最优策略下的回收收益: {optimal_minus}")

```

1.4 问题三代码

Listing 7: 枚举法求解

```

import itertools
import numpy as np
from functools import reduce
import operator
import matplotlib.pyplot as plt
from scipy.stats import linregress

# 参数定义
N = 100 # 生产量
sell_price = 200 # 成品卖出价格
per_assembly_cost = 8 # 装配成本
per_semi_final_disassembly_cost = 6 # 半成品拆解成本
per_final_disassembly_cost = 10 # 成品拆解成本
per_replacement_cost = 40 # 调换损失

# 零配件1至8 半成品1至3 成品的次品率
p_bad = 0.1

# 零配件1至8的购买单价
parts_prices = [2, 8, 12, 2, 8, 12, 8, 12]

# 零配件1至8的检测成本
parts_testing_costs = [1, 1, 2, 1, 1, 2, 1, 2]

# 半成品和成品的检测成本
semi_final_testing_cost = 4
final_testing_cost = 6

# 利润计算函数

```

```

def calculate_profit(N, parts_prices, parts_testing_costs, semi_final_testing_cost,
    final_testing_cost,
        per_assembly_cost, per_semi_final_disassembly_cost,
        per_final_disassembly_cost,
        per_replacement_cost, sell_price, p_bad, detect_parts,
        detect_semi_finals, disassemble_semi_finals, detect_final,
        disassemble_final):

    # 初始化利润
    profit = 0

    # 计算半成品个数
    p_final_1 = p_final_2 = 1 - (1 - (1 - p_bad) ** 4) * detect_semi_finals[0]
    p_final_3 = 1 - (1 - (1 - p_bad) ** 3) * detect_semi_finals[1]

    num_semi_1 = num_semi_2 = N / p_final_1
    num_semi_3 = N / p_final_3

    # 计算零配件个数
    num_1 = num_4 = num_semi_1 / (1 - p_bad * detect_parts[0])
    num_2 = num_5 = num_semi_1 / (1 - p_bad * detect_parts[1])
    num_3 = num_6 = num_semi_1 / (1 - p_bad * detect_parts[2])
    num_7 = num_semi_3 / (1 - p_bad * detect_parts[3])
    num_8 = num_semi_3 / (1 - p_bad * detect_parts[4])

    # 成品回收均值
    E_revenue_final = 2 * (1 - (1 - p_bad) ** 4) / (1 - (1 - p_bad) ** 12) * (60 +
        detect_parts[0] * parts_testing_costs[0]) + \
        detect_parts[1] * parts_testing_costs[1] + detect_parts[2] *
        parts_testing_costs[2] + \
        (1 - (1 - p_bad) ** 3) / (1 - (1 - p_bad) ** 12) * (28 + detect_parts[3] *
        parts_testing_costs[3] + detect_parts[4] * parts_testing_costs[4])

    # 半成品1和2的回收均值
    E_revenue_semi_1 = E_revenue_semi_2 = (p_bad / (1 - (1 - p_bad) ** 3) * (1 - p_bad)) * (2 +
        8 + 12)
    E_revenue_semi_3 = (p_bad / (1 - (1 - p_bad) * (1 - p_bad)) * (1 - p_bad)) * (8 + 12)

    # 零配件采购总成本
    total_parts_cost = sum(num * price for num, price in zip([num_1, num_2, num_3, num_4,
        num_5, num_6, num_7, num_8], parts_prices))

    # 零配件检测总成本
    total_parts_testing_cost = 2 / (1 - p_bad) * num_semi_1 * sum(detect_parts[i] *
        parts_testing_costs[i] for i in range(3)) + \
        1 / (1 - p_bad) * num_semi_3 * sum(detect_parts[i] *
        parts_testing_costs[i] for i in range(3, 5))

```

```

# 单个半成品1和2的成本
semi_prices_1 = semi_prices_2 = per_assembly_cost + parts_prices[0] + parts_prices[1] +
    parts_prices[2] + \
        sum(detect_parts[i] * parts_testing_costs[i] for i in range(3))
semi_prices_3 = per_assembly_cost + parts_prices[6] + parts_prices[7] + detect_parts[3] *
    parts_testing_costs[3] + \
        detect_parts[4] * parts_testing_costs[4]

# 半成品合格率
P1 = P2 = (1 - p_bad * (1 - detect_semi_finals[0])) * reduce(operator.mul, ((1 - p_bad * (1
    - detect_parts[i])) for i in range(3)))
P3 = (1 - p_bad * (1 - detect_parts[3])) * (1 - p_bad * (1 - detect_parts[4])) * (1 - p_bad
    * (1 - detect_semi_finals[1]))

# 半成品的装配+检测+替换成本
total_semi = per_assembly_cost * (2 * num_semi_1 + num_semi_3) + \
    semi_final_testing_cost * (2 * num_semi_1 * detect_semi_finals[0] + num_semi_3 *
        detect_semi_finals[1]) + \
        2 * (1 - detect_semi_finals[0]) * num_semi_1 * (1 - P1) * semi_prices_1 + \
        (1 - detect_semi_finals[1]) * num_semi_3 * (1 - P3) * semi_prices_3

# 成品合格率
final_P = (1 - P1 * (1 - detect_semi_finals[0])) * (1 - P1 * (1 - detect_semi_finals[0])) *
    (1 - P3 * (1 - detect_semi_finals[1])) * (1 - p_bad * (1 - detect_final))

# 成品检测成本
total_final_testing_cost = N * detect_final * final_testing_cost

# 成品装配成本
total_assembly_cost = N * per_assembly_cost

# 成品调换损失
replacement_cost = (per_replacement_cost + 2 * semi_prices_1 + semi_prices_3 +
    per_assembly_cost) * final_P * N * (1 - detect_final)

# 成品的总成本
total_final = total_final_testing_cost + total_assembly_cost + replacement_cost

# 拆解成本
disassembly_cost = 2 * disassemble_semi_finals[0] * num_semi_1 * (1 - P1) *
    per_semi_final_disassembly_cost + \
        disassemble_semi_finals[1] * num_semi_3 * (1 - P3) *
        per_semi_final_disassembly_cost + \
        disassemble_final * N * (1 - final_P) * per_final_disassembly_cost

# 销售额

```

```

revenue = sell_price * N * (1 - p_bad * (1-(1-detect_semi_finals[0] * P1)) *
    (1-(1-detect_semi_finals[0] * P1)) * (1-(1-detect_semi_finals[1]) * P3))

# 回收利润
recycle = E_revenue_final * N * disassemble_final * (1 - final_P) + \
    E_revenue_semi_1 * num_semi_1 * disassemble_semi_finals[0] * (1 - P1) * 2 + \
    E_revenue_semi_3 * num_semi_3 * disassemble_semi_finals[1] * (1 - P3)

# 总利润计算
profit = revenue + recycle - total_parts_testing_cost - total_parts_cost - total_semi -
    total_final - disassembly_cost

return profit

# 动态规划表
states = list(itertools.product([0, 1], repeat=11)) # 5个零件检测决策 + 2个半成品检测决策 +
    2个半成品拆解决策 + 1个成品检测决策 + 1个成品拆解决策

ANSWER = np.zeros(len(states))

# 动态规划计算
for idx, state in enumerate(states):
    detect_parts = state[:5] # 零件12378是否检测
    detect_semi_finals = state[5:7] # 半成品13是否检测
    disassemble_semi_finals = state[7:9] # 是否拆解半成品13
    detect_final, disassemble_final = state[9:11] # 成品是否检测, 是否拆解
    ANSWER[idx] = calculate_profit(
        N, parts_prices, parts_testing_costs, semi_final_testing_cost, final_testing_cost,
        per_assembly_cost, per_semi_final_disassembly_cost, per_final_disassembly_cost,
        per_replacement_cost, sell_price, p_bad, detect_parts,
        detect_semi_finals, disassemble_semi_finals, detect_final, disassemble_final
    )

# 找最优策略
optimal_strategy = max(states, key=lambda state: ANSWER[list(states).index(state)])

print("最优策略: ")
for i, decision in enumerate(optimal_strategy):
    print(f"第{i+1}个决策 (0为不检测/不拆解, 1为检测/拆解) : {decision}")

# 计算总成本
optimal_profit = ANSWER[list(states).index(optimal_strategy)]
print(f"最优策略下的总利润: {optimal_profit}")

```

Listing 8: 遗传算法求解

```

import random
import numpy as np
from functools import reduce
import operator

# 参数定义
N = 100 # 生产量
sell_price = 200 # 成品卖出价格
per_assembly_cost = 8 # 装配成本
per_semi_final_disassembly_cost = 6 # 半成品拆解成本
per_final_disassembly_cost = 10 # 成品拆解成本
per_replacement_cost = 40 # 调换损失

# 零配件1至8 半成品1至3 成品的次品率
p_bad = 0.1

# 零配件1至8的购买单价
parts_prices = [2, 8, 12, 2, 8, 12, 8, 12]

# 零配件1至8的检测成本
parts_testing_costs = [1, 1, 2, 1, 1, 2, 1, 2]

# 半成品和成品的检测成本
semi_final_testing_cost = 4
final_testing_cost = 6

# 利润计算函数
def calculate_profit(N, parts_prices, parts_testing_costs, semi_final_testing_cost,
                    final_testing_cost,
                    per_assembly_cost, per_semi_final_disassembly_cost,
                    per_final_disassembly_cost,
                    per_replacement_cost, sell_price, p_bad, detect_parts,
                    detect_semi_finals, disassemble_semi_finals, detect_final,
                    disassemble_final):

    # 初始化利润
    profit = 0

    # 计算半成品个数
    p_final_1 = p_final_2 = 1 - (1 - 0.9 * 0.9 * 0.9 * 0.9) * detect_semi_finals[0]
    p_final_3 = 1 - (1 - 0.9 * 0.9 * 0.9) * detect_semi_finals[1]

    num_semi_1 = num_semi_2 = N / p_final_1
    num_semi_3 = N / p_final_3

    # 计算零配件个数

```



```

num_1 = num_4 = num_semi_1 / (1 - p_bad * detect_parts[0])
num_2 = num_5 = num_semi_1 / (1 - p_bad * detect_parts[1])
num_3 = num_6 = num_semi_1 / (1 - p_bad * detect_parts[2])
num_7 = num_semi_3 / (1 - p_bad * detect_parts[3])
num_8 = num_semi_3 / (1 - p_bad * detect_parts[4])

# 零配件采购总成本
total_parts_cost = sum(num * price for num, price in zip([num_1, num_2, num_3, num_4,
num_5, num_6, num_7, num_8], parts_prices))

# 零配件检测总成本
total_parts_testing_cost = 2 / 0.9 * num_semi_1 * sum(detect_parts[i] *
parts_testing_costs[i] for i in range(3)) + \
1 / 0.9 * num_semi_3 * sum(detect_parts[i] * parts_testing_costs[i]
for i in range(3, 5))

# 半成品合格率
P1 = P2 = (1 - p_bad * (1 - detect_semi_finals[0])) * reduce(operator.mul, ((1 - p_bad * (1
- detect_parts[i])) for i in range(3)))
P3 = (1 - p_bad * (1 - detect_parts[3])) * (1 - p_bad * (1 - detect_parts[4])) * (1 - p_bad
* (1 - detect_semi_finals[1]))

# 单个半成品1和2的成本
semi_prices_1 = semi_prices_2 = per_assembly_cost + parts_prices[0] + parts_prices[1] +
parts_prices[2] + \
sum(detect_parts[i] * parts_testing_costs[i] for i in range(3))
semi_prices_3 = per_assembly_cost + parts_prices[6] + parts_prices[7] + detect_parts[3] *
parts_testing_costs[3] + \
detect_parts[4] * parts_testing_costs[4]

# 半成品的装配+检测+替换成本
total_semi = per_assembly_cost * (2 * num_semi_1 + num_semi_3) + \
semi_final_testing_cost * (2 * num_semi_1 * detect_semi_finals[0] + num_semi_3 *
detect_semi_finals[1]) + \
2 * (1 - detect_semi_finals[0]) * num_semi_1 * (1 - P1) * semi_prices_1 + \
(1 - detect_semi_finals[1]) * num_semi_3 * (1 - P3) * semi_prices_3

# 成品合格率
final_P = (1 - P1 * (1 - detect_semi_finals[0])) * (1 - P1 * (1 - detect_semi_finals[0])) *
(1 - P3 * (1 - detect_semi_finals[1])) * (1 - p_bad * (1 - detect_final))

# 成品检测成本
total_final_testing_cost = N * detect_final * final_testing_cost

# 成品装配成本
total_assembly_cost = N * per_assembly_cost

# 成品调换损失

```

```

replacement_cost = (per_replacement_cost + 2 * semi_prices_1 + semi_prices_3 +
                    per_assembly_cost) * final_P * N * (1 - detect_final)

# 成品的总成本
total_final = total_final_testing_cost + total_assembly_cost + replacement_cost

# 销售额
revenue = sell_price * N * (1 - p_bad * (1-(1-detect_semi_finals[0] * P1)) *
                          (1-(1-detect_semi_finals[0] * P1)) * (1-(1-detect_semi_finals[1]) * P3))

# 成品回收均值
E_revenue_final = 2 * 0.3439 / 0.71757 * (60 + detect_parts[0] * parts_testing_costs[0]) + \
                  detect_parts[1] * parts_testing_costs[1] + detect_parts[2] *
                  parts_testing_costs[2] + \
                  0.271 / 0.71757 * (28 + detect_parts[3] * parts_testing_costs[3] +
                  detect_parts[4] * parts_testing_costs[4])

# 半成品1和2的回收均值
E_revenue_semi_1 = E_revenue_semi_2 = (p_bad / (1 - (1 - p_bad) * (1 - p_bad) * (1 -
p_bad)) * (1 - p_bad)) * (2 + 8 + 12)
E_revenue_semi_3 = (p_bad / (1 - (1 - p_bad) * (1 - p_bad)) * (1 - p_bad)) * (8 + 12)

# 回收利润
recycle = E_revenue_final * N * disassemble_final * (1 - final_P) + \
          E_revenue_semi_1 * num_semi_1 * disassemble_semi_finals[0] * (1 - P1) * 2 + \
          E_revenue_semi_3 * num_semi_3 * disassemble_semi_finals[1] * (1 - P3)

# 拆解成本
disassembly_cost = (2 * disassemble_semi_finals[0] * num_semi_1 * (1 - P1) *
                    per_semi_final_disassembly_cost +
                    disassemble_semi_finals[1] * num_semi_3 * (1 - P3) *
                    per_semi_final_disassembly_cost +
                    disassemble_final * N * (1 - final_P) * per_final_disassembly_cost)

# 总利润计算
profit = revenue + recycle - total_parts_testing_cost - total_parts_cost - total_semi -
        total_final - disassembly_cost

return profit

# 遗传算法参数
POPULATION_SIZE = 100 # 种群大小
GENERATIONS = 100 # 代数
MUTATION_RATE = 0.1 # 变异概率
CROSSOVER_RATE = 0.7 # 交叉概率

# 初始化种群
population = [list(np.random.randint(2, size=11)) for _ in range(POPULATION_SIZE)]

```

```

# 适应度函数
def fitness(individual):
    detect_parts = individual[:5] # 零件12378是否检测
    detect_semi_finals = individual[5:7] # 半成品13是否检测
    disassemble_semi_finals = individual[7:9] # 是否拆解半成品13
    detect_final, disassemble_final = individual[9:11] # 成品是否检测, 是否拆解
    return calculate_profit(
        N, parts_prices, parts_testing_costs, semi_final_testing_cost, final_testing_cost,
        per_assembly_cost, per_semi_final_disassembly_cost, per_final_disassembly_cost,
        per_replacement_cost, sell_price, p_bad, detect_parts,
        detect_semi_finals, disassemble_semi_finals, detect_final, disassemble_final
    )

# 选择函数
def selection(population, fitness_values):
    for i in range(len(fitness_values)):
        if fitness_values[i] < 0:
            fitness_values[i] = 0
    p = np.array(fitness_values) / np.sum(fitness_values)

    select = np.random.choice(len(population), size=2, replace=False, p=p)
    selected_individuals = [population[i] for i in select]
    return selected_individuals

# 交叉函数
def crossover(parent1, parent2):
    if random.random() < CROSSTOVER_RATE:
        point = random.randint(1, len(parent1)-2)
        child1 = parent1[:point] + parent2[point:]
        child2 = parent2[:point] + parent1[point:]
    else:
        child1, child2 = parent1, parent2
    return child1, child2

# 变异函数
def mutation(child):
    for i in range(len(child)):
        if random.random() < MUTATION_RATE:
            child[i] = 1 - child[i] # 翻转基因
    return child

# 主循环
for generation in range(GENERATIONS):
    fitness_values = [fitness(individual) for individual in population]
    new_population = []

```

```

# 选择、交叉、变异
while len(new_population) < POPULATION_SIZE:
    parent1, parent2 = selection(population, fitness_values)
    child1, child2 = crossover(parent1, parent2)
    child1 = mutation(child1)
    child2 = mutation(child2)
    new_population.extend([child1, child2])

population = new_population

# 最优解
best_individual = max(population, key=fitness)
best_fitness = fitness(best_individual)

print("最优策略: ")
for i, decision in enumerate(best_individual):
    print(f"第{i+1}个决策 (0为不检测/不拆解, 1为检测/拆解) : {decision}")

print(f"最优策略下的总利润: {best_fitness}")

```

1.5 问题四代码

Listing 9: 贝叶斯 + 蒙特卡洛模拟

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import beta, binom

# 贝叶斯估计修正次品率函数
def bayesian_update(alpha, beta_prior, X, n):
    """
    使用贝叶斯更新公式计算次品率的后验均值和后验分布
    alpha: Beta分布的先验参数
    beta_prior: Beta分布的先验参数
    X: 抽样得到的次品数
    n: 抽样的总样本数
    """
    alpha_post = alpha + X
    beta_post = beta_prior + n - X
    posterior_mean = alpha_post / (alpha_post + beta_post) # 计算更新后的均值
    return posterior_mean, alpha_post, beta_post

# 蒙特卡洛模拟函数
def monte_carlo_simulation(alpha_prior, beta_prior, true_defect_rate, n, num_simulations):
    """
    进行蒙特卡洛模拟, 生成新的后验分布, 并记录每次迭代的后验均值
    """

```

```

alpha_prior: Beta分布的先验参数
beta_prior: Beta分布的先验参数
true_defect_rate: 真正的次品率, 用于生成模拟样本数据
n: 每次模拟的样本总数
num_simulations: 模拟的次数
"""

posterior_means = []
observed_defect_rates = [] # 记录每次模拟的次品率
alpha_post, beta_post = alpha_prior, beta_prior

for _ in range(num_simulations):
    # 生成模拟的样本数据, 模拟检测n个样本, 基于真实次品率生成次品数
    X = binom.rvs(n, true_defect_rate)
    observed_defect_rates.append(X / n) # 记录每次抽样的次品率

    # 使用贝叶斯更新得到新的后验均值
    posterior_mean, alpha_post, beta_post = bayesian_update(alpha_post, beta_post, X, n)

    # 记录每次模拟后的后验均值
    posterior_means.append(posterior_mean)

return posterior_means, observed_defect_rates, alpha_post, beta_post

# 绘制单个先验和后验分布, 并显示蒙特卡洛模拟结果
def plot_single_prior_posterior(alpha_prior, beta_prior, true_defect_rate, n, num_simulations):
    # 生成样本
    X = binom.rvs(n, true_defect_rate)

    # 贝叶斯更新, 获取后验分布
    posterior_mean, alpha_post, beta_post = bayesian_update(alpha_prior, beta_prior, X, n)

    # 蒙特卡洛模拟
    posterior_means, observed_defect_rates, alpha_post_mc, beta_post_mc =
        monte_carlo_simulation(alpha_prior, beta_prior, true_defect_rate, n, num_simulations)

    # 计算更新后的均值
    updated_mean = alpha_post / (alpha_post + beta_post)

    # 绘制先验分布和后验分布在同一张图上
    x = np.linspace(0, 1, 100)
    prior_pdf = beta.pdf(x, alpha_prior, beta_prior)
    posterior_pdf = beta.pdf(x, alpha_post, beta_post)

    plt.figure(figsize=(5, 5))

    # 先验分布
    plt.plot(x, prior_pdf, label=f'先验分布  = {alpha_prior},  = {beta_prior}', linestyle='--')

```

```

# 后验分布
plt.plot(x, posterior_pdf, label=f'后验分布 = {alpha_post}, = {beta_post}')

# 绘制多个观测点
plt.scatter(observed_defect_rates, [0] * len(observed_defect_rates), s=1, c='r',
            marker='o', label='观测次品率')

plt.xlabel('次品率')
plt.ylabel('概率密度')
plt.title(f'先验和后验分布 (模拟真实次品率={true_defect_rate:.3f},
            更新后的次品率={updated_mean:.3f})')
plt.legend()
plt.show()

# 绘制蒙特卡洛模拟结果
plt.figure()
mc_x = range(1, num_simulations + 1)
plt.plot(mc_x, posterior_means, label='蒙特卡洛后验均值')
plt.title(f'蒙特卡洛模拟后验均值 (更新后的次品率={updated_mean:.3f})')
plt.xlabel('模拟次数')
plt.ylabel('后验均值')
plt.legend()

plt.tight_layout()
plt.show()

# 设置参数
alpha_prior = 3 # 先验 参数, 表示先验信念中的“成功”次数
beta_prior = 27 # 先验 参数, 表示先验信念中的“失败”次数
n = 43 # 每次模拟的样本数
num_simulations = 100 # 蒙特卡洛模拟次数

# 设置多个 true_defect_rate, 从 18% 到 22%, 步长为 0.5%
true_defect_rates = np.arange(0.18, 0.225, 0.005)

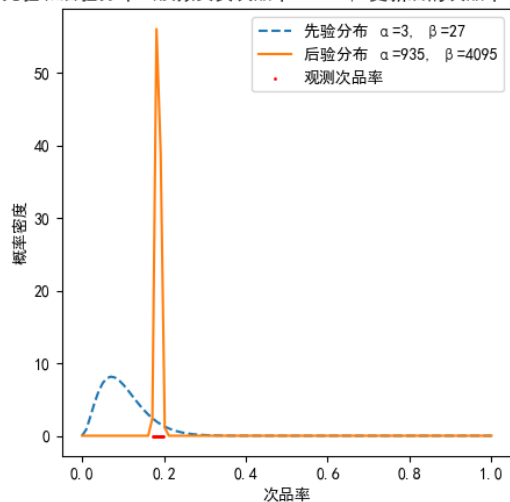
# 绘制不同 true_defect_rate 下的先验和后验分布, 分别画图
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

for true_defect_rate in true_defect_rates:
    plot_single_prior_posterior(alpha_prior, beta_prior, true_defect_rate, n, num_simulations)

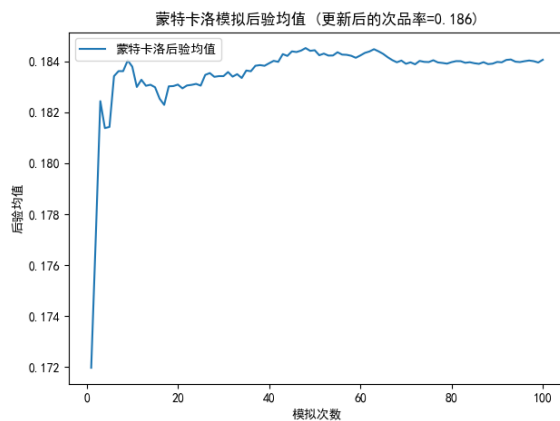
```

B 数据图片

先验和后验分布 (模拟真实次品率=0.185, 更新后的次品率=0.186)



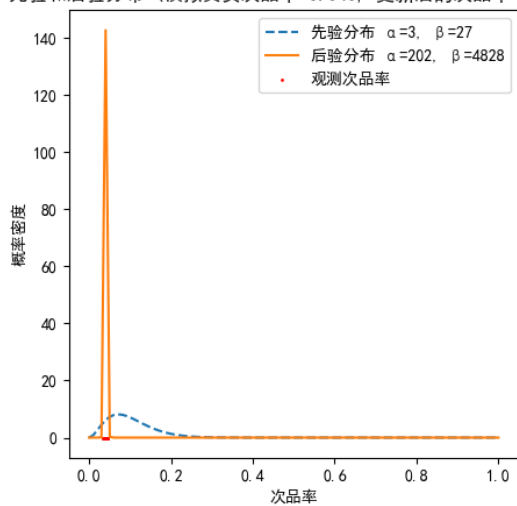
(a) 20% 先验与后验次品率的对比



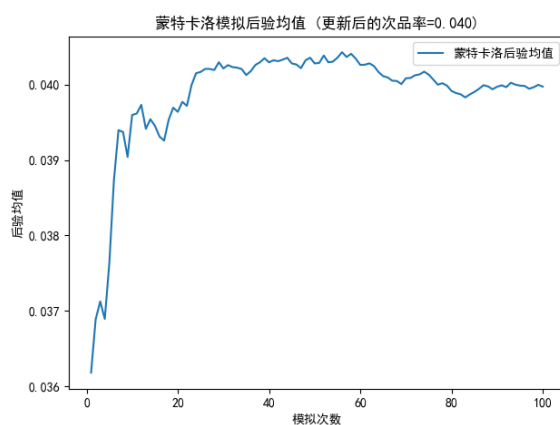
(b) 蒙特卡洛模拟后验均值

图 11 20% 先验值下的蒙特卡洛模拟

先验和后验分布 (模拟真实次品率=0.040, 更新后的次品率=0.040)



(a) 5% 先验与后验次品率的对比



(b) 蒙特卡洛模拟后验均值

图 12 5% 先验值下的蒙特卡洛模拟